



School of Computer Science and Electronic Engineering

MSc Cyber Security

Academic Year 2026-2027

Explainable Blockchain and AI Misbehaviour Detection for IoT and UAV Networks with LLM Assisted Interpretation

A project report submitted by: Yousef Hesham Mosaad Abdelghani - 6738880

A project supervised by: Mohammad Shojafar

A report submitted in partial fulfilment of the requirement for the degree of Master of
Science.

University of Surrey
School of Computer Science and Electronic Engineering
Guildford, Surrey GU2 7XH
United Kingdom.
Tel: +44 (0)1483 300800

Abstract

IoT and UAV systems increasingly mediate safety relevant processes, yet because they communicate over shared and often unauthenticated media a single misbehaving node can inject falsified telemetry or control frames and undermine any decision built on that data. This dissertation designs and empirically evaluates an explainable framework that unites three defences the literature has largely pursued in isolation. The first being a blockchain ledger that keeps records tamper evident, the second classical machine learning that detects malicious traffic, and thirdly a large language model that interprets each decision. A critical review distilled three persistent weaknesses into five research questions where the weaknesses include consensus mechanisms being chosen from general claims rather than measured on real traffic, learned detection being divorced from the ledger meant to protect it, and the resulting verdicts staying opaque.

To address these research questions, every record from two open and labelled datasets (a UAV and a IoT dataset) was recast as a ledger transaction and driven through one reproducible Python pipeline: a two layer smart contract, a consensus layer, an AI detector, an explainability layer and an LLM stage. Eight consensus algorithms, ten detectors and the LIME and Kernel SHAP explainers were built from first principles and measured under controlled and seeded conditions across both domains.

Ledger integrity proved to be dictated by protocol design rather than the traffic itself as Byzantine and authority based protocols blocked every attack, whereas a crash fault tolerant protocol admitted roughly 8% at a fraction of the cost, confirming that tolerating crashes is not the same as resisting adversaries. Deterministic rules alone caught almost nothing making a learned policy indispensable, and tree ensembles balanced missed attacks against wrongly rejected records best reaching perfect accuracy once evidence was pooled at block level. Explanation then overturned an intuitive assumption revealing that the UAV attacks inject protocol conforming payloads rather than flooding the bus, a result that justifies the whole architecture. Finally, small locally hosted models produced faithful justifications and valid candidate rules, though retrieval augmentation merely added latency without measurable benefit, bounding a prior published finding to particular model scales and tasks. Together the work shows that measuring security, trust and interpretability as a single pipeline exposes conclusions that studying any part alone would miss.

Acknowledgments

I am grateful to my supervisor Dr. Shojafar for teaching me so much and guiding me through this project, it would not have been possible without him. Additionally, I would like to dedicate this work to my family who have supported me both emotionally and financially throughout my higher education.

I certify that the work presented in the dissertation is my own unless referenced.

Signature:

A handwritten signature in black ink, appearing to be 'Y. Shojafar', written over a horizontal line.

Date: 30/08/2026

Total number of pages: 69

Total number of words: 15,115

Table of Contents

Abstract	2
Acknowledgments	3
Abbreviations	7
List of Tables.....	8
TABLE 1: CONSENSUS KPI'S	8
TABLE 2: AI KPI'S (BOTH DATASETS)	8
TABLE 3: BLOCKCHAIN KPI'S	9
TABLE 4: ROC AREA UNDER CURVE	9
TABLE 5: LLM CODE CORECTNESS, BERTSCORE & MEAN RESPONSE TIME	10
List of Figures	11
FIGURE 1: CONSENSUS KPI'S	11
FIGURE 2: EXPLAINABILITY (LIME & SHAP) DATA.....	14
FIGURE 3: CONFUSION MATRICIES	17
FIGURE 4: ROC CURVES.....	21
FIGURE 5: ABNORMAL BLOCK DETECTION	23
FIGURE 6: LLM DATA (RAG & NO-RAG)	23
CHAPTER 1: INTRODUCTION	25
1.1 Background	25
1.2 Research Aim and Objectives.....	27
1.3 Project Approach	28
1.4 Contributions	29
1.5 Structure of the Dissertation	29
CHAPTER 2: LITERATURE REVIEW	31
2.1 Blockchain for IoT and the IoD	31
2.2 Consensus Mechanisms for Distributed Ledgers	32
2.3 AI Based Misbehaviour and Intrusion Detection.....	33
2.4 Explainable AI for Security	33
2.5 LLMs for Security Interpretation and Mitigation.....	34

2.6 Critical Comparison and Research Gaps	35
2.7 Summary.....	37
CHAPTER 3: RESEARCH APPROACH	39
3.1 Choice of Research Methodology	39
3.2 Application of the Methodology to This Project	40
3.3 Data Access and Preparation	41
3.4 Ethical and Responsible Research Considerations	41
3.5 Summary.....	42
CHAPTER 4: METHODOLOGY.....	43
4.1 System Design	43
4.1.1 Requirements Analysis.....	43
4.1.2 Architecture and Threat Model	44
4.1.3 Data Design: Datasets as Transaction Workloads	44
4.1.4 Smart Contract Design	45
4.1.5 Consensus Layer Design	45
4.1.6 AI Misbehaviour Detection Design	46
4.1.7 Explainability Design	47
4.1.8 LLM Assisted Interpretation Design	47
4.2 System Development	48
4.2.1 Development Environment and Tools	48
4.2.2 Increment One: Data Ingestion and Transaction Mapping	48
4.2.3 Increment Two: Consensus and Smart Contract Implementation	48
4.2.4 Increment Three: Feature Engineering and Detector Training	49
4.2.5 Increment Four: From First Principles LIME and Kernel SHAP	49
4.2.6 Increment Five: LLM Assisted Stage Implementation	50
4.2.7 Reproducibility and Validation	50
4.3 NextGCom 2026 Conference Paper	51
4.4 Summary.....	51
CHAPTER 5: TESTING, RESULTS AND EVALUATION	53

5.1 Testing and Verification Approach.....	53
5.2 Consensus and Smart Contract Results	54
5.3 AI Misbehaviour Detection Results	55
5.4 Explainability and the Key Interpretive Finding	55
5.5 LLM Assisted Interpretation Results.....	56
5.6 Comparison with Existing Research.....	57
5.7 Critical Evaluation of Results	58
5.8 Critical Evaluation of Project Objectives	58
5.9 Summary.....	59
CHAPTER 6: CONCLUSION.....	61
6.1 Summary of the Dissertation.....	61
6.2 Research Contributions.....	62
6.3 Limitations and Future Research and Development	63
6.4 Personal Reflections	64
References	65
Ethical Approval.....	67
Appendix.....	68

Abbreviations

- IoT – Internet of Things
- UAV – Unmanned Ariel Vehicles
- AI – Artificial Intelligence
- LLM – Large Language Models
- IoD – Internet of Drones
- RAG – Retrieval Augmented Generation
- CRISP-DM – Cross Industry Standard Process for Data Mining
- RQ – Research Question
- ROC curve – Receiver Operating Characteristic curve
- FNR – False Negative Rate
- FPR – False Positive Rate
- KPI – KEY Performance Indicator
- LIME – Local Interpretable Model-agnostic Explanations
- SHAP – SHapley Additive exPlanations
- PoW – Proof of Work
- PoA – Proof of Authority
- PoC – Proof of Capacity
- PoS – Proof of Stake
- DPoS – Delegated Proof of Stake
- PBFT – Practical Byzantine Fault Tolerance
- dBFT – Delegated Byzantine Fault Tolerance

List of Tables

TABLE 1: CONSENSUS KPI'S

TABLE 1.2 – Measured consensus KPIs (averaged over seeds)
 Key: Integrity = % attacks kept off-ledger (higher=better) | Decentralisation = 1-Gini (higher=better)
 Energy = modelled relative units | all other KPIs measured

dataset	IoT					UAV				
algorithm	Block time (ms)	Decentralisation	Energy (model)	Integrity %	Msgs/block	Block time (ms)	Decentralisation	Energy (model)	Integrity %	Msgs/block
PoW	6.87	0.669	4150.0	88.2	49.0	7.23	0.669	4150.0	88.4	49.0
PoS	0.16	0.695	65.0	98.2	65.0	0.11	0.695	65.0	98.2	65.0
DPOS	0.26	0.711	21.0	96.7	21.0	0.21	0.711	21.0	96.7	21.0
PoA	0.16	0.917	23.2	100.0	23.0	0.11	0.917	23.2	100.0	23.0
PoC	0.24	0.659	99.0	88.3	49.0	0.19	0.659	99.0	88.2	49.0
Raft	0.15	0.679	12.7	92.1	13.0	0.10	0.679	12.7	91.9	13.0
PBFT	0.15	0.100	189.1	100.0	189.0	0.10	0.100	189.1	100.0	189.0
dBFT	0.16	0.048	860.0	100.0	860.0	0.11	0.048	860.0	100.0	860.0

TABLE 2: AI KPI'S (BOTH DATASETS)

TABLE 2.2 – AI KPIs, IoT dataset (sorted by F1)

	Accuracy	Precision	Recall	F1	ROC-AUC	MCC
Model						
Extra Trees	0.809	0.809	0.891	0.848	0.882	0.599
Random Forest	0.806	0.806	0.889	0.845	0.894	0.591
Decision Tree	0.797	0.801	0.878	0.837	0.815	0.571
Hist GBM	0.777	0.756	0.925	0.832	0.880	0.534
LDA	0.738	0.705	0.963	0.814	0.813	0.464
Logistic Regression	0.729	0.702	0.949	0.807	0.814	0.438
Linear SVM	0.727	0.701	0.947	0.806	0.810	0.433
Gaussian NB	0.704	0.668	1.000	0.801	0.806	0.421
k-NN	0.758	0.805	0.783	0.794	0.873	0.500
AdaBoost	0.683	0.653	1.000	0.790	0.799	0.373

TABLE 2.2 – AI KPIs, UAV dataset (sorted by F1)

	Accuracy	Precision	Recall	F1	ROC-AUC	MCC
Model						
Hist GBM	0.972	0.981	0.943	0.961	0.997	0.940
Random Forest	0.964	0.962	0.938	0.950	0.993	0.922
Extra Trees	0.960	0.961	0.928	0.944	0.989	0.913
Decision Tree	0.953	0.936	0.935	0.936	0.949	0.898
k-NN	0.923	0.937	0.846	0.889	0.967	0.832
AdaBoost	0.915	0.906	0.857	0.881	0.971	0.816
Gaussian NB	0.470	0.408	1.000	0.580	0.582	0.258
Logistic Regression	0.684	0.602	0.404	0.483	0.769	0.279
Linear SVM	0.686	0.609	0.399	0.482	0.769	0.283
LDA	0.685	0.613	0.381	0.470	0.765	0.278

TABLE 3: BLOCKCHAIN KPI'S

TABLE 2.3 – Blockchain KPIs (both datasets)

Key: FNR = attacks admitted to ledger (security risk, lower=better) | FPR = valid records rejected (availability cost, lower=better)

	IoT FNR	IoT FPR	IoT Latency(us)	UAV FNR	UAV FPR	UAV Latency(us)
Extra Trees	10.9%	31.2%	14.8	7.2%	2.2%	14.8
Random Forest	11.1%	31.7%	12.6	6.2%	2.2%	16.2
Decision Tree	12.2%	32.3%	0.2	6.5%	3.7%	0.1
Hist GBM	7.5%	44.2%	14.3	5.7%	1.1%	10.2
LDA	3.7%	59.7%	0.2	61.9%	13.9%	0.1
Logistic Regression	5.1%	59.7%	0.3	59.6%	15.4%	0.1
Linear SVM	5.3%	59.7%	0.1	60.1%	14.8%	0.1
Gaussian NB	0.0%	73.5%	0.5	0.0%	83.7%	0.4
k-NN	21.7%	28.0%	314.3	15.4%	3.3%	170.0
AdaBoost	0.0%	78.7%	3.7	14.3%	5.1%	3.6

TABLE 4: ROC AREA UNDER CURVE

TABLE 2.4 – ROC Area Under Curve (AUC) by model and dataset (sorted by mean)

	IoT	UAV	Mean
Random Forest	0.894	0.993	0.943
Hist GBM	0.880	0.997	0.938
Extra Trees	0.882	0.989	0.935
k-NN	0.873	0.967	0.920
AdaBoost	0.799	0.971	0.885
Decision Tree	0.815	0.949	0.882
Logistic Regression	0.814	0.769	0.791
Linear SVM	0.810	0.769	0.790
LDA	0.813	0.765	0.789
Gaussian NB	0.806	0.582	0.694

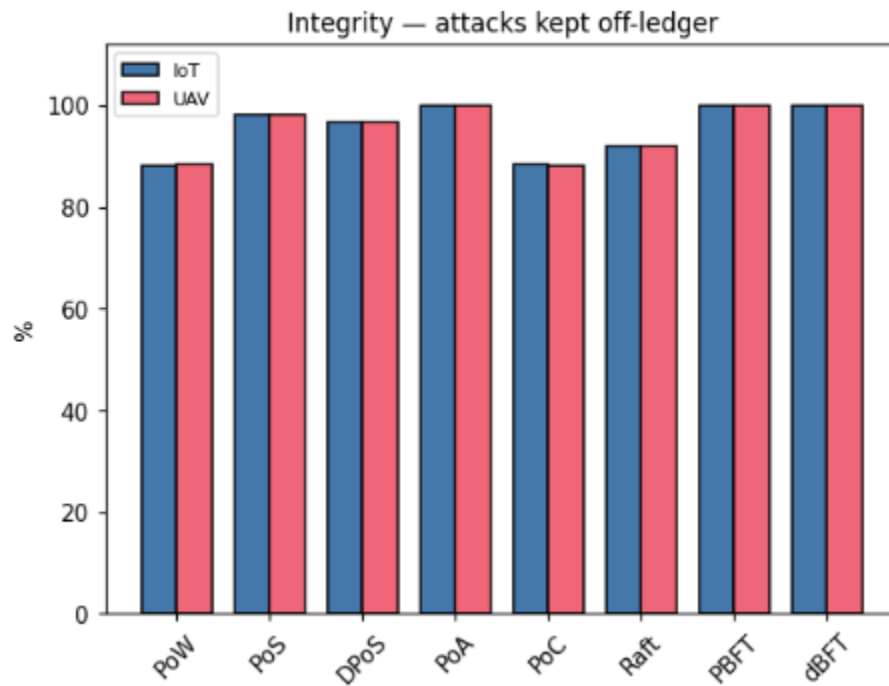
TABLE 5: LLM CODE CORECTNESS, BERTSCORE & MEAN RESPONSE TIME

TABLE 3.1 – LLM interpretation quality (mean ± 95% CI over sampled attacks, both datasets)			
Modified Code Correctness			
	gemma2:2b	llama3.2	qwen2.5:3b
No-RAG	0.950 ± 0.051	0.950 ± 0.051	0.933 ± 0.056
RAG	0.917 ± 0.058	0.900 ± 0.076	0.917 ± 0.058
BERTScore			
	gemma2:2b	llama3.2	qwen2.5:3b
No-RAG	0.819 ± 0.006	0.839 ± 0.003	0.835 ± 0.004
RAG	0.823 ± 0.004	0.838 ± 0.004	0.842 ± 0.005
Average Response Time (s)			
	gemma2:2b	llama3.2	qwen2.5:3b
No-RAG	8.280 ± 1.486	5.737 ± 0.775	12.229 ± 1.706
RAG	8.123 ± 0.650	8.126 ± 1.118	14.105 ± 2.264

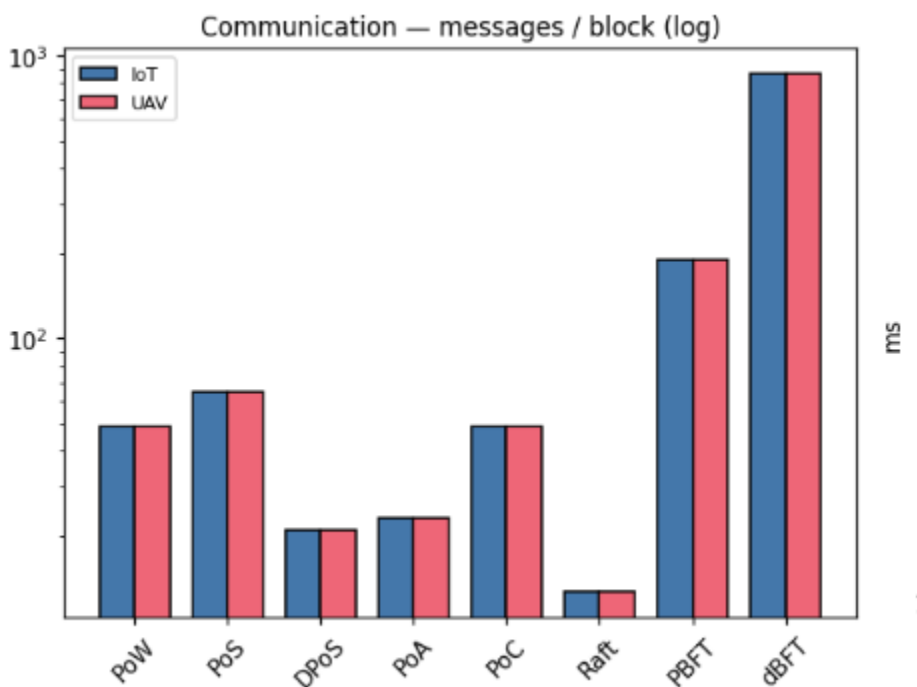
List of Figures

FIGURE 1: CONSENSUS KPI'S

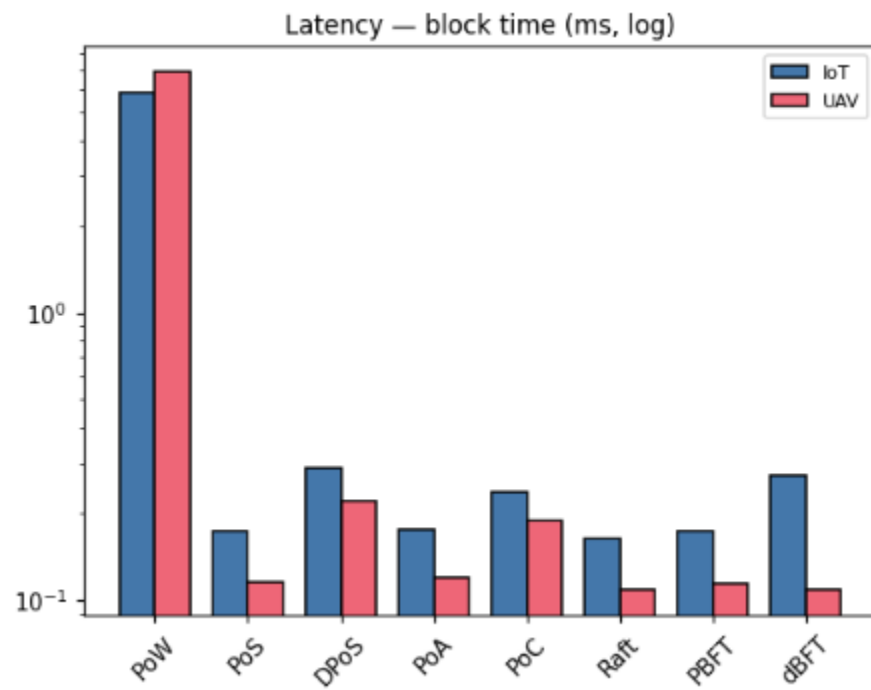
1.1: INTEGRITY



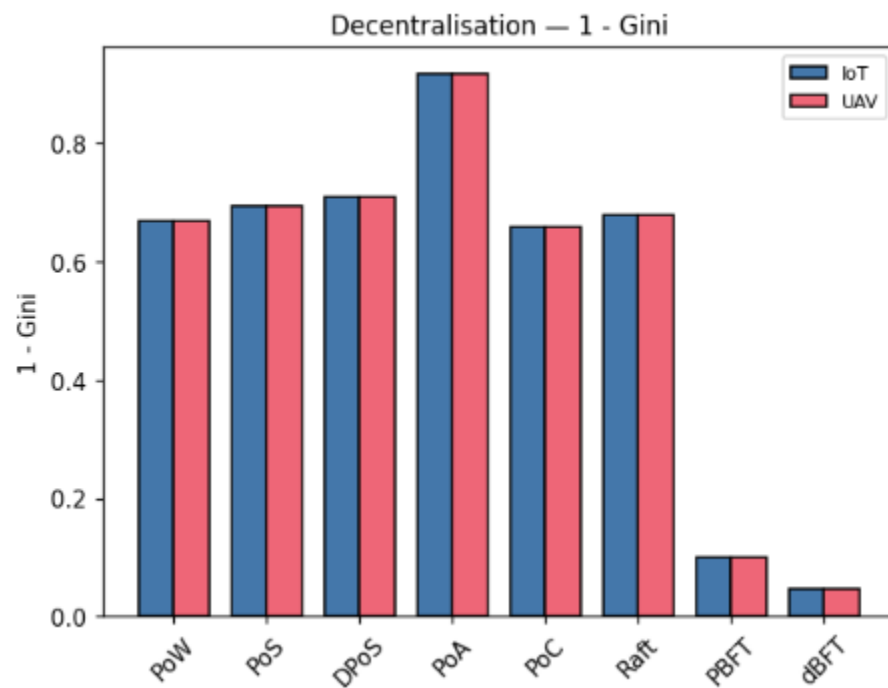
1.2: COMMUNICATION



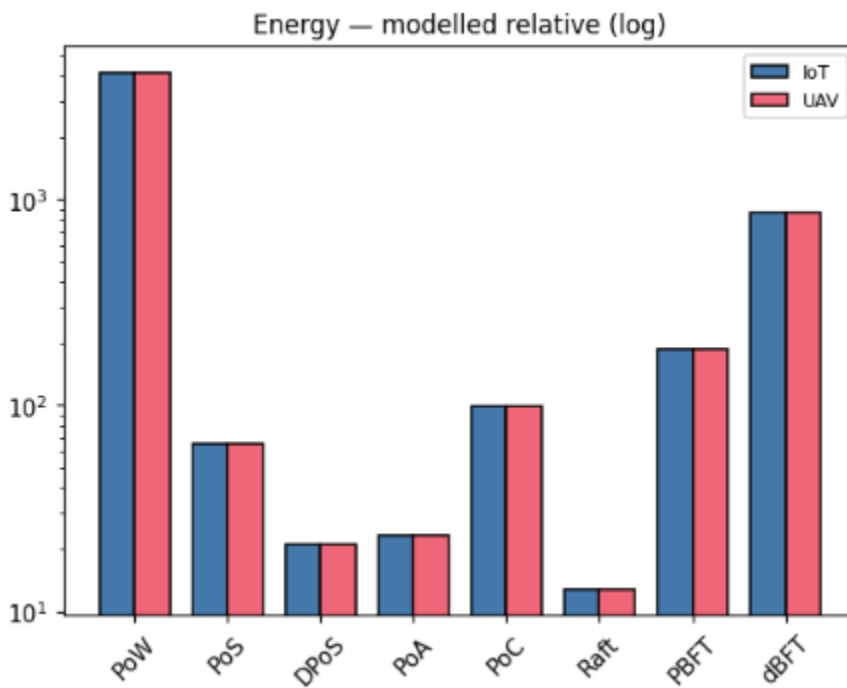
1.3: LATENCY



1.4: DECENTRALISATION



1.5: ENERGY



1.6: TRADEOFF

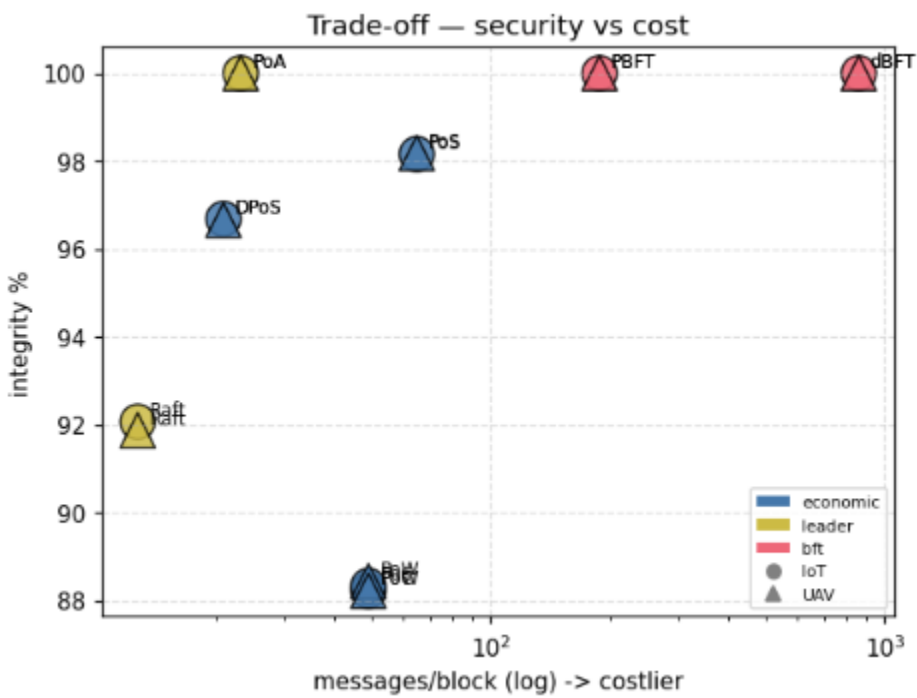
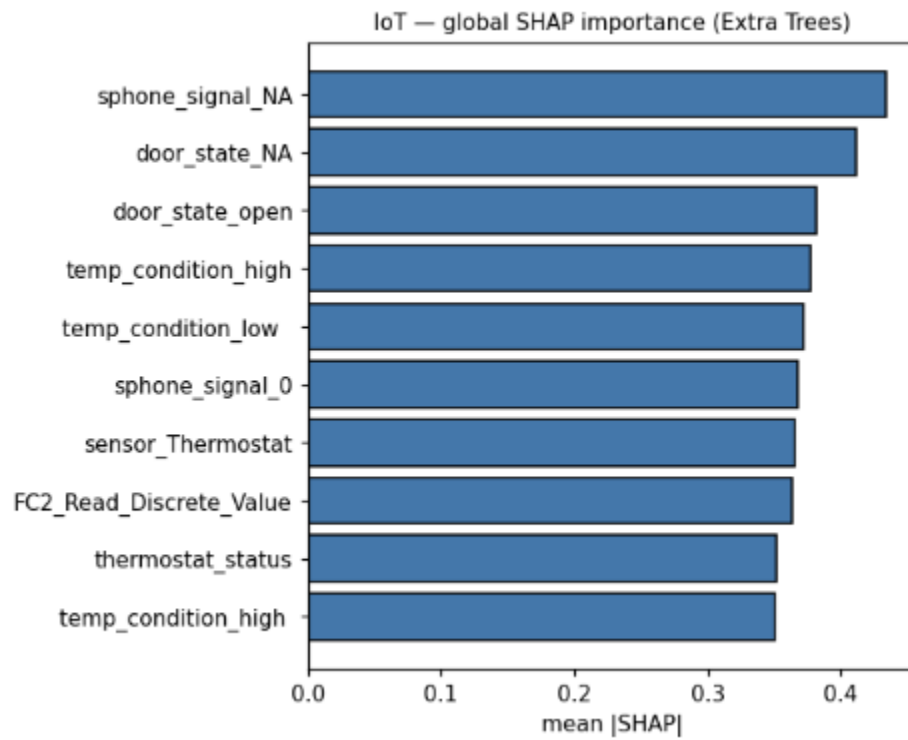
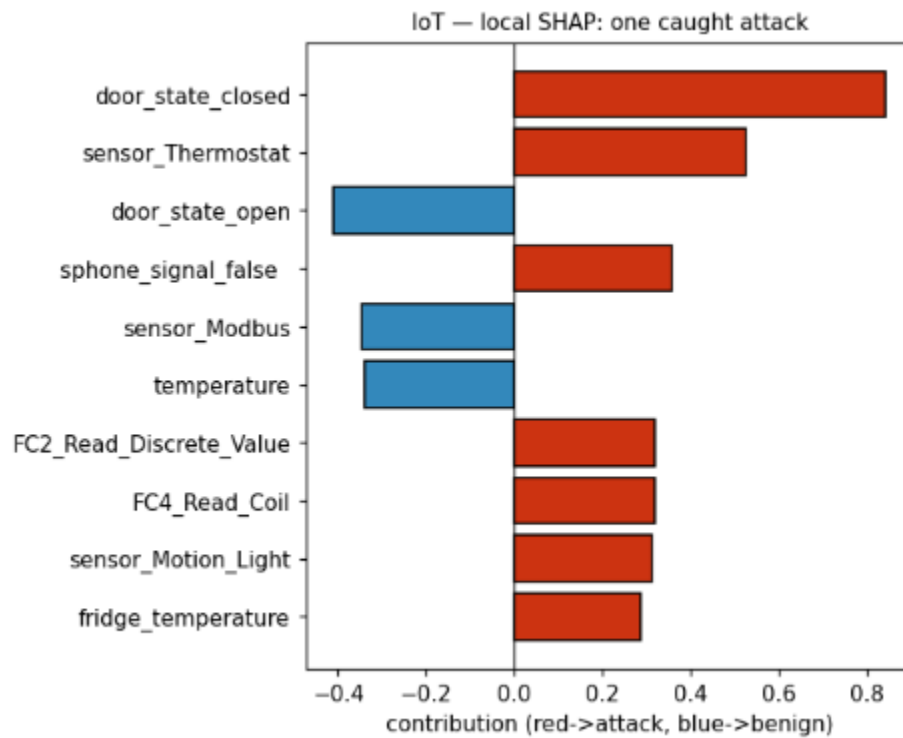


FIGURE 2: EXPLAINABILITY (LIME & SHAP) DATA

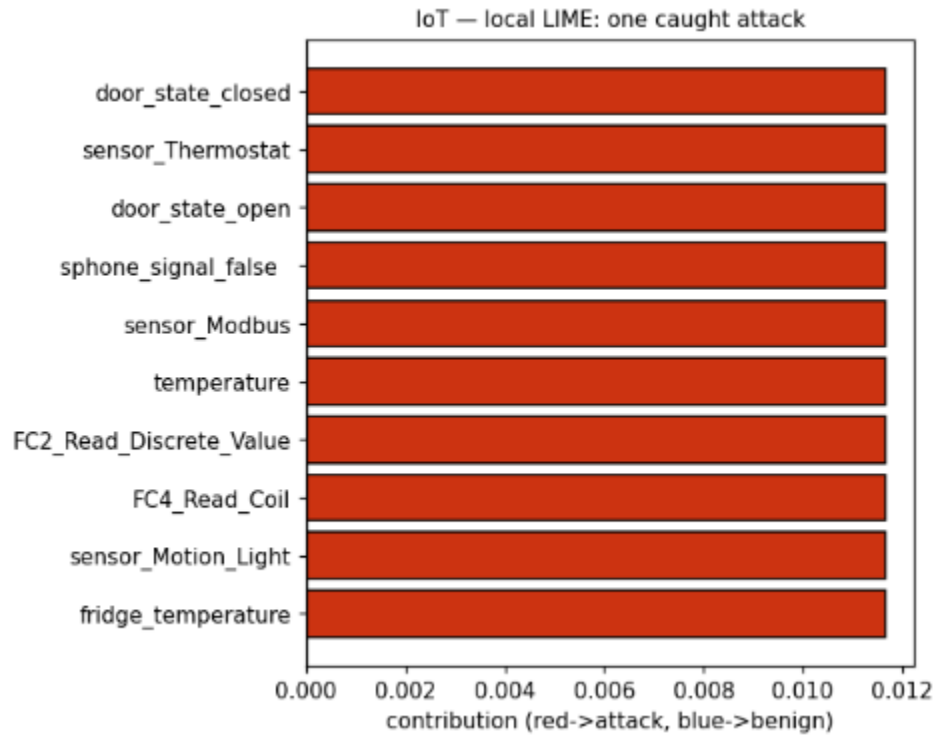
2.1:



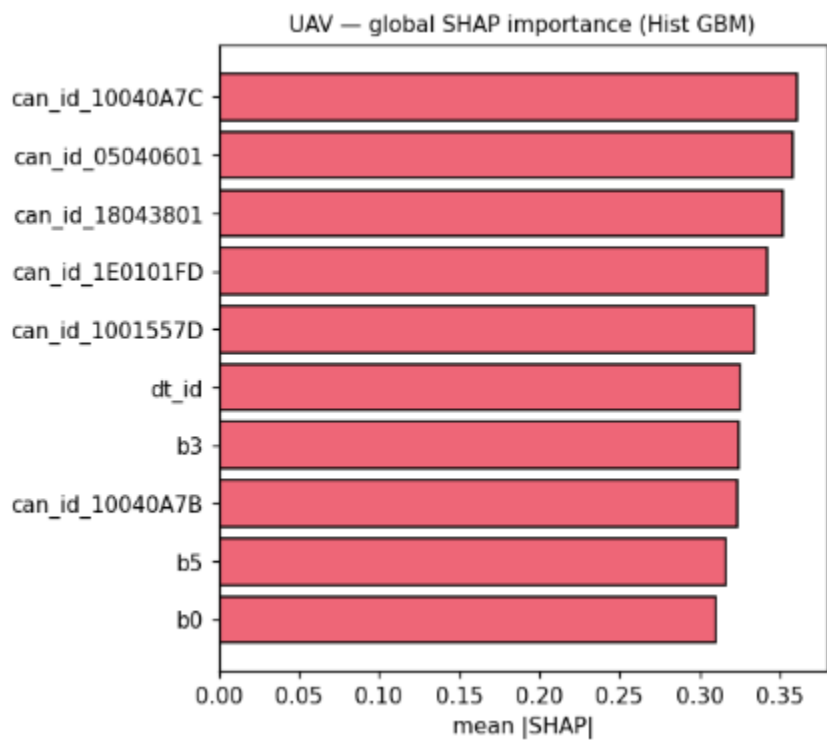
2.2:



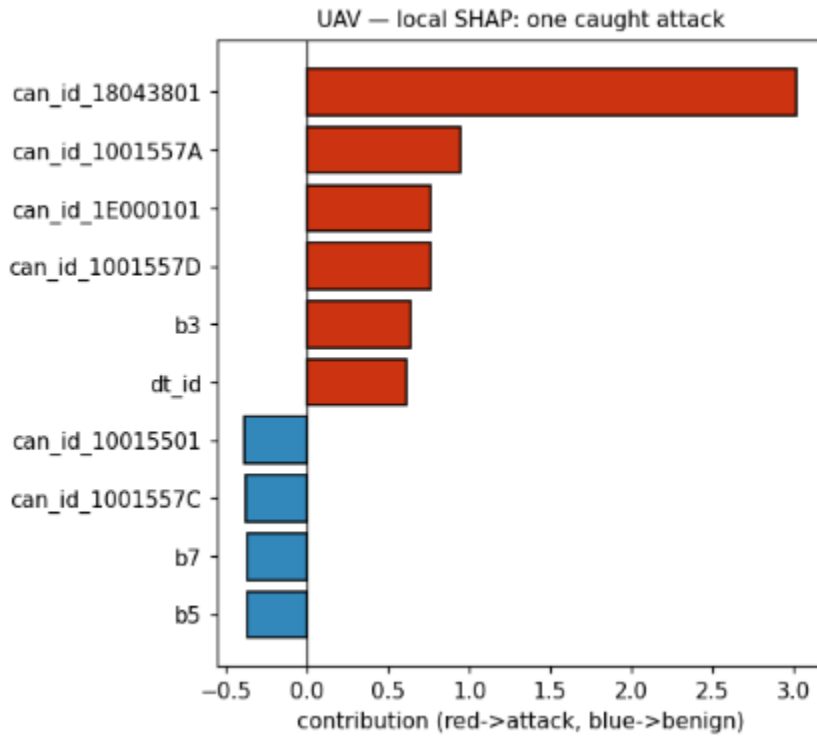
2.3:



2.4:



2.5:



2.6

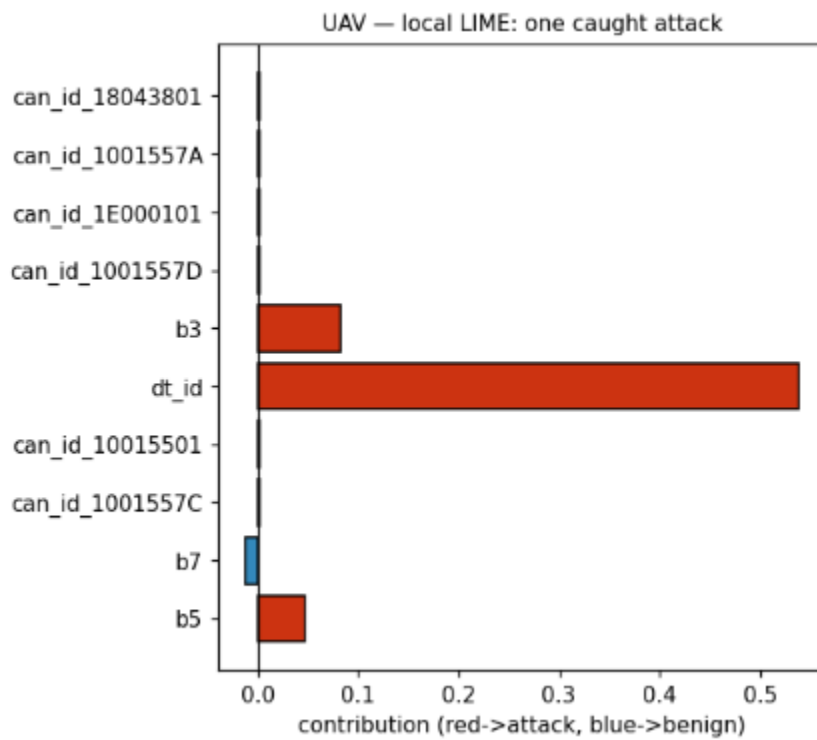
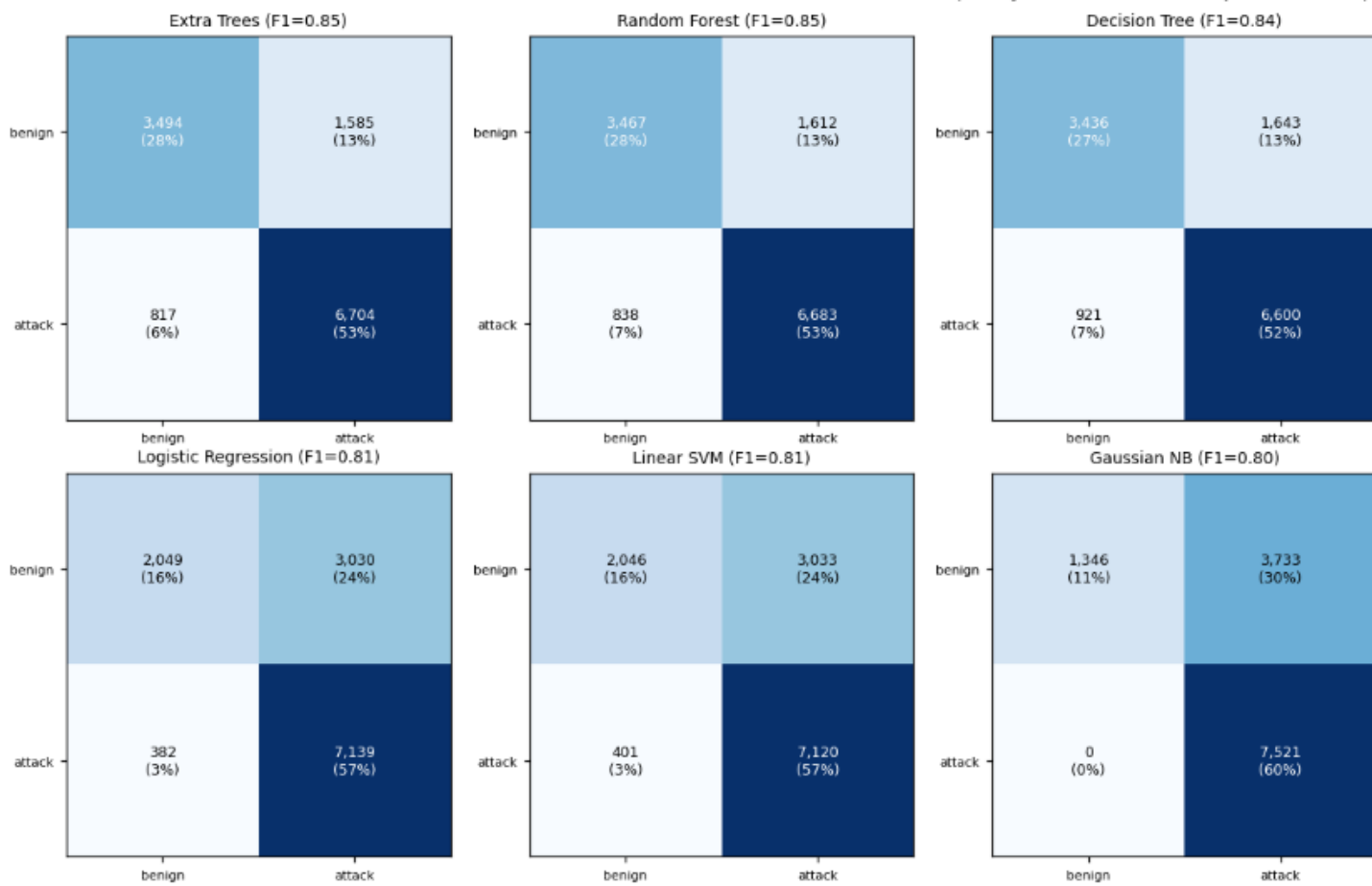
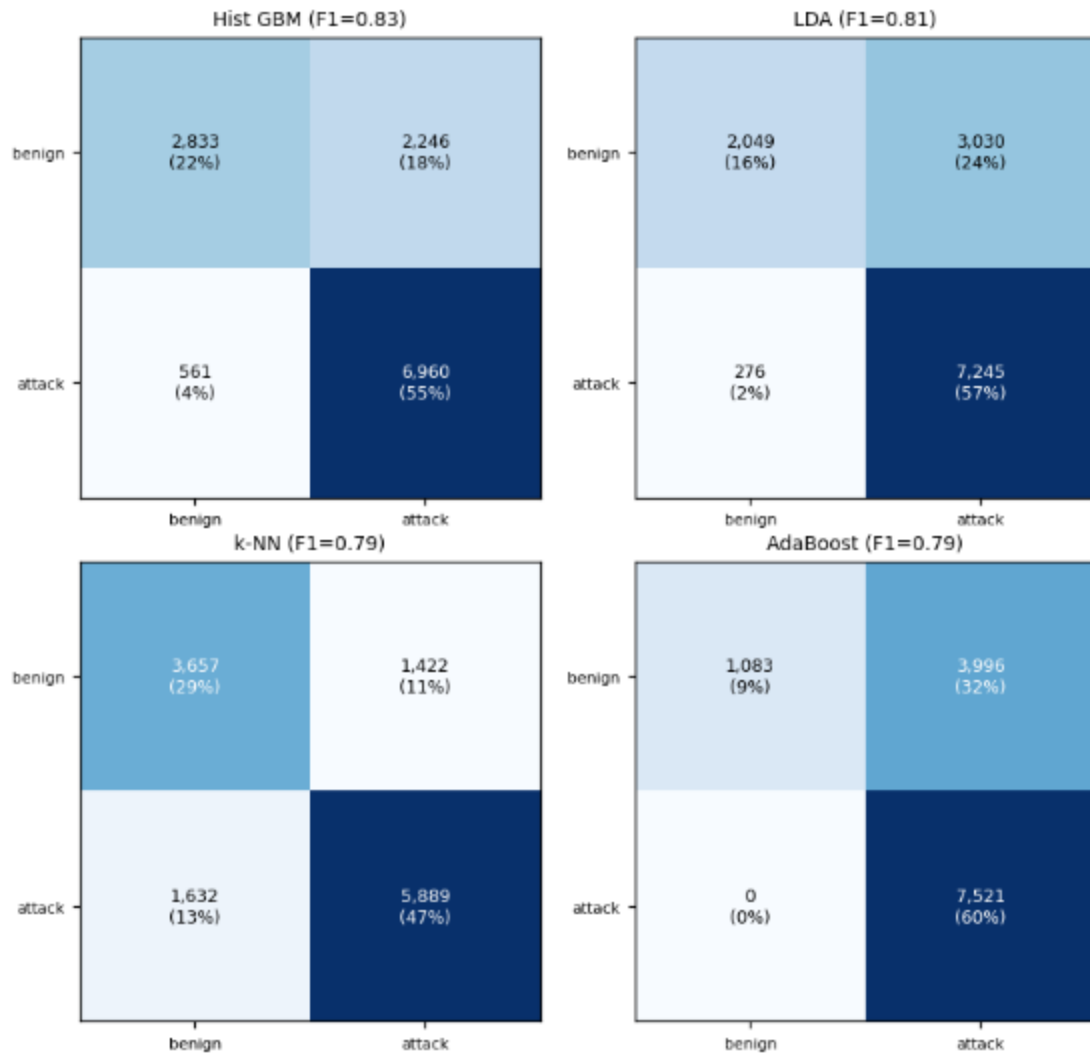


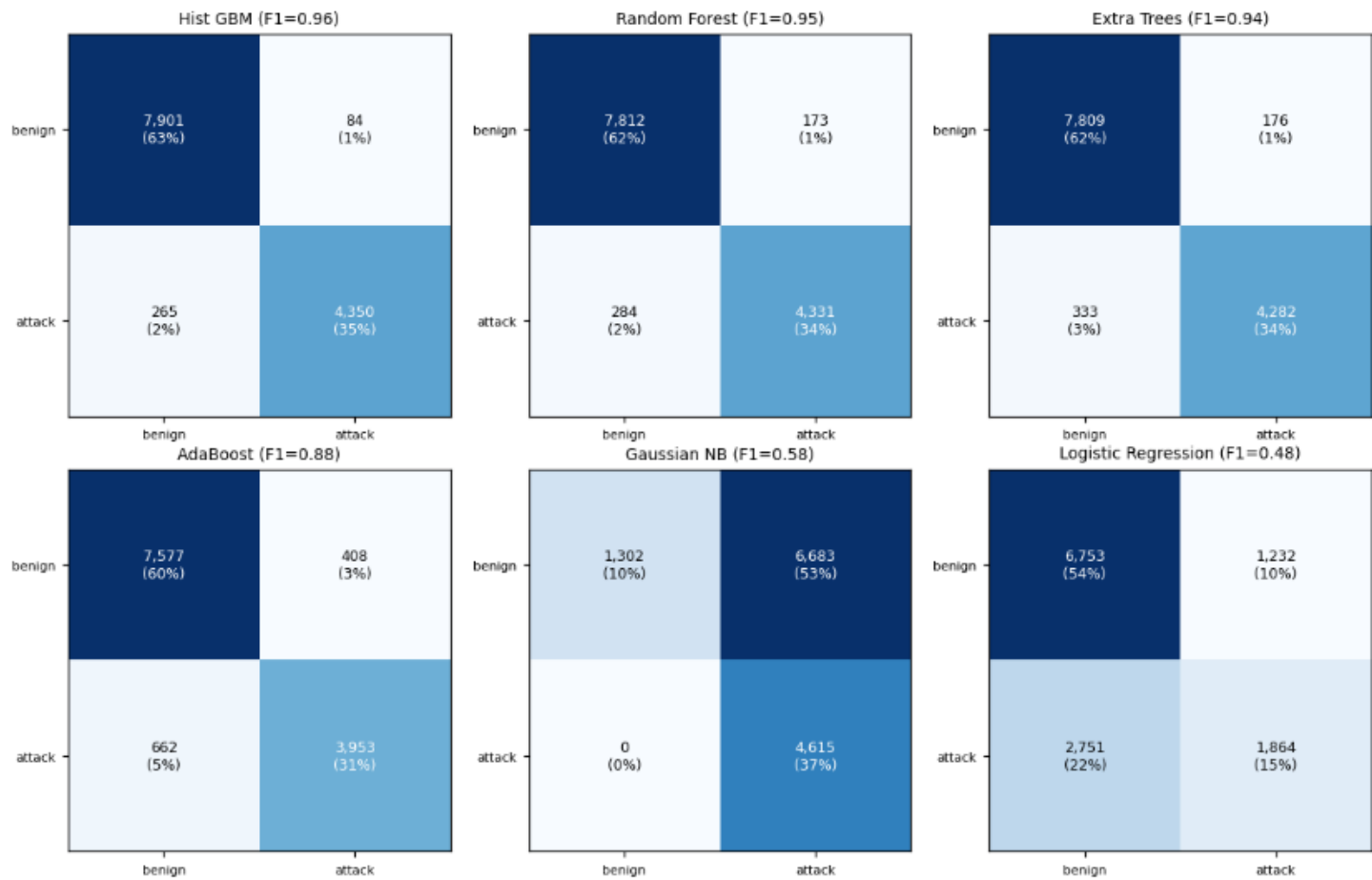
FIGURE 3: CONFUSION MATRICES

3.1 – IOT DATASET ALL AI MODELS





3.2 – UAV DATASET ALL AI MODELS



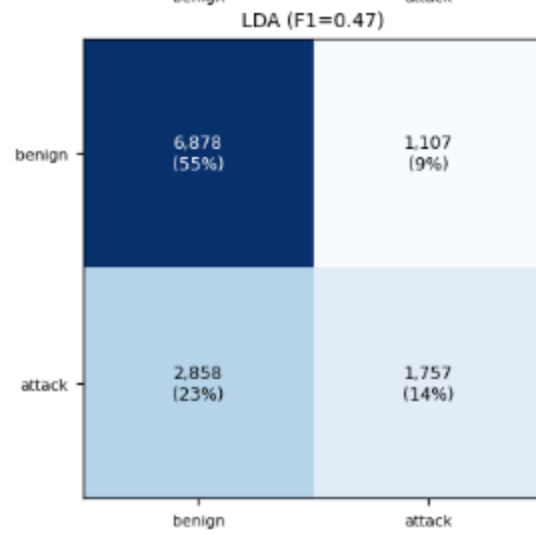
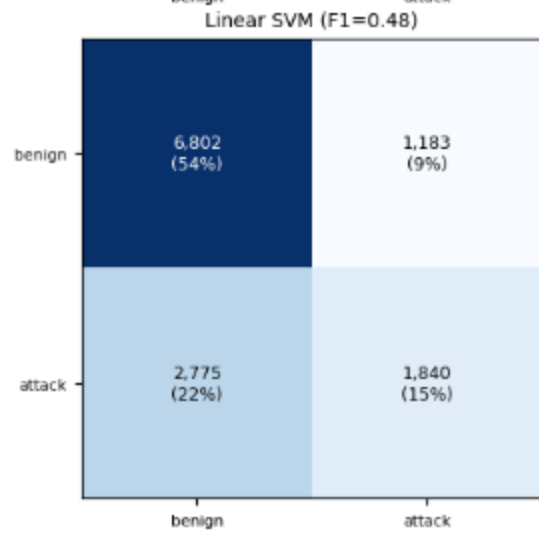
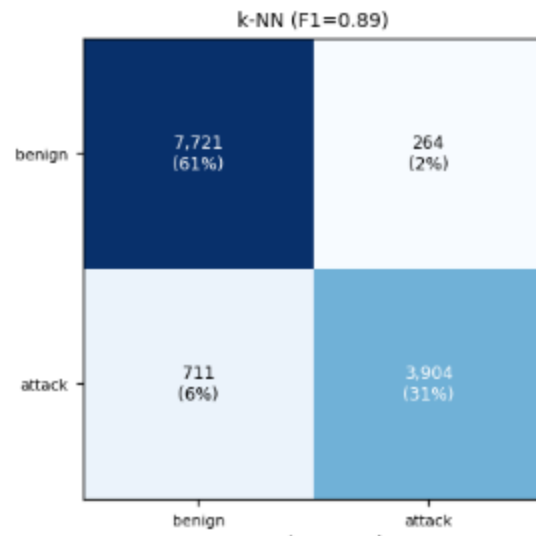
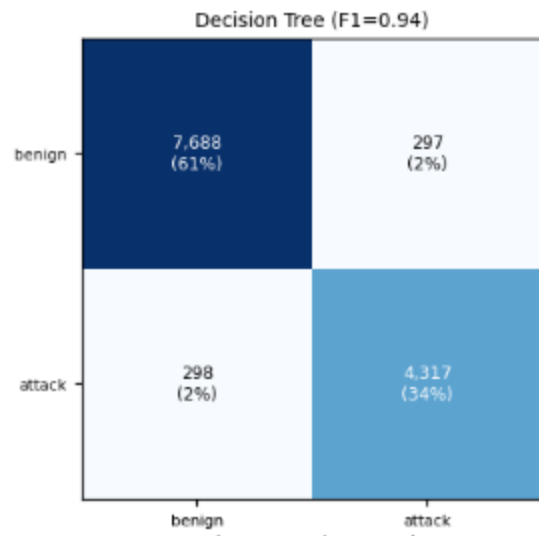
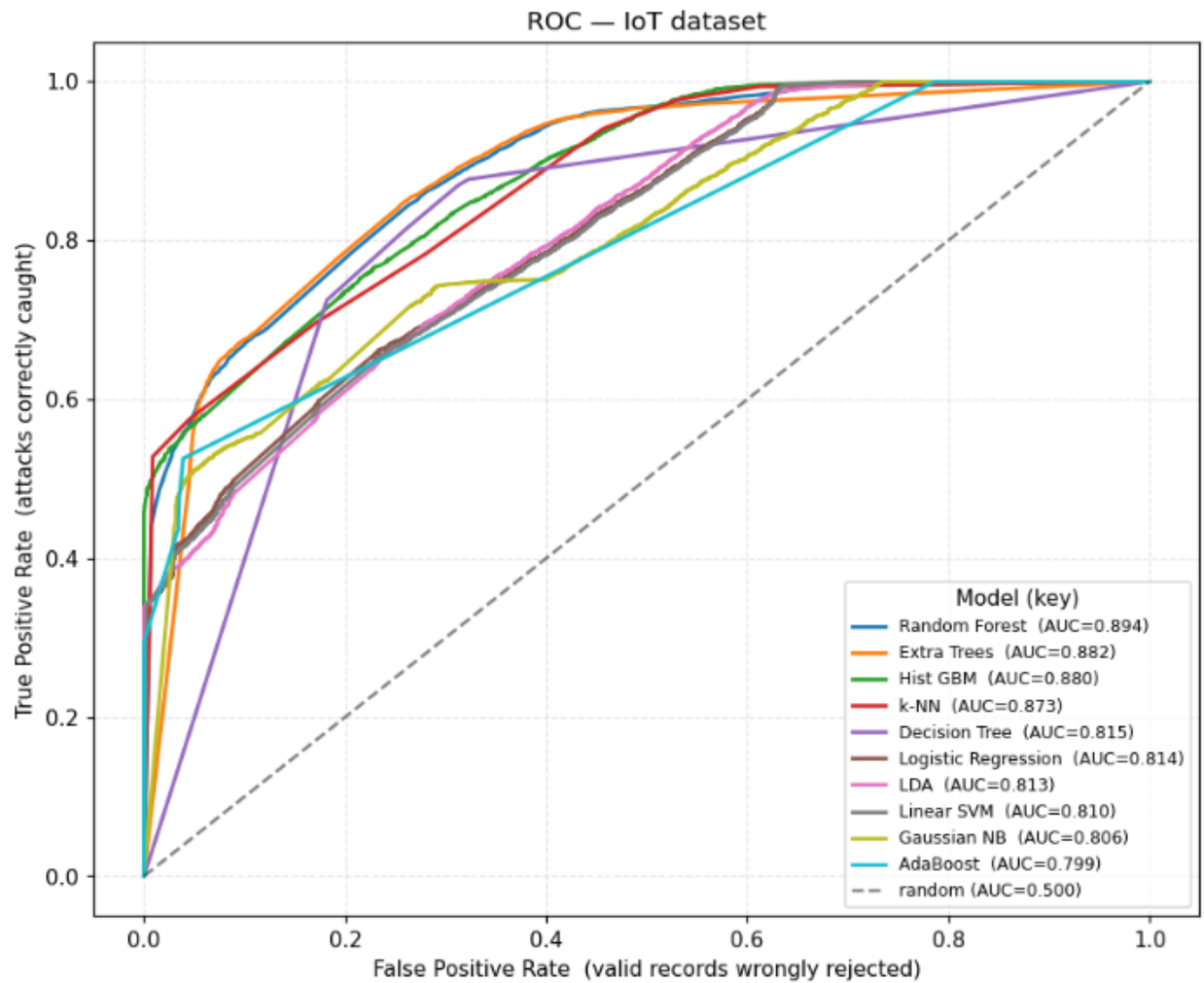


FIGURE 4: ROC CURVES

4.1 – IOT DATASET



4.2 – UAV DATASET

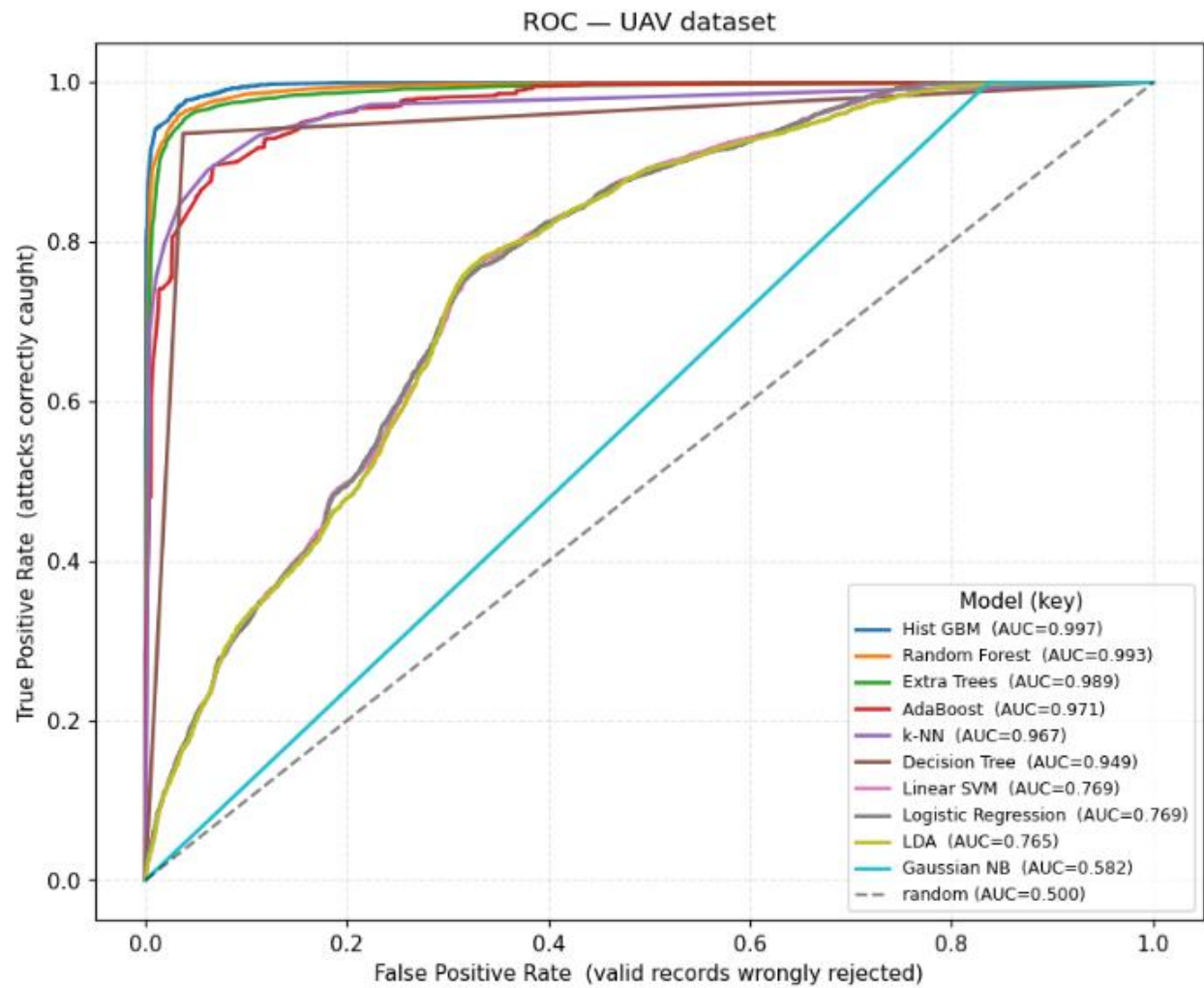


FIGURE 5: ABNORMAL BLOCK DETECTION

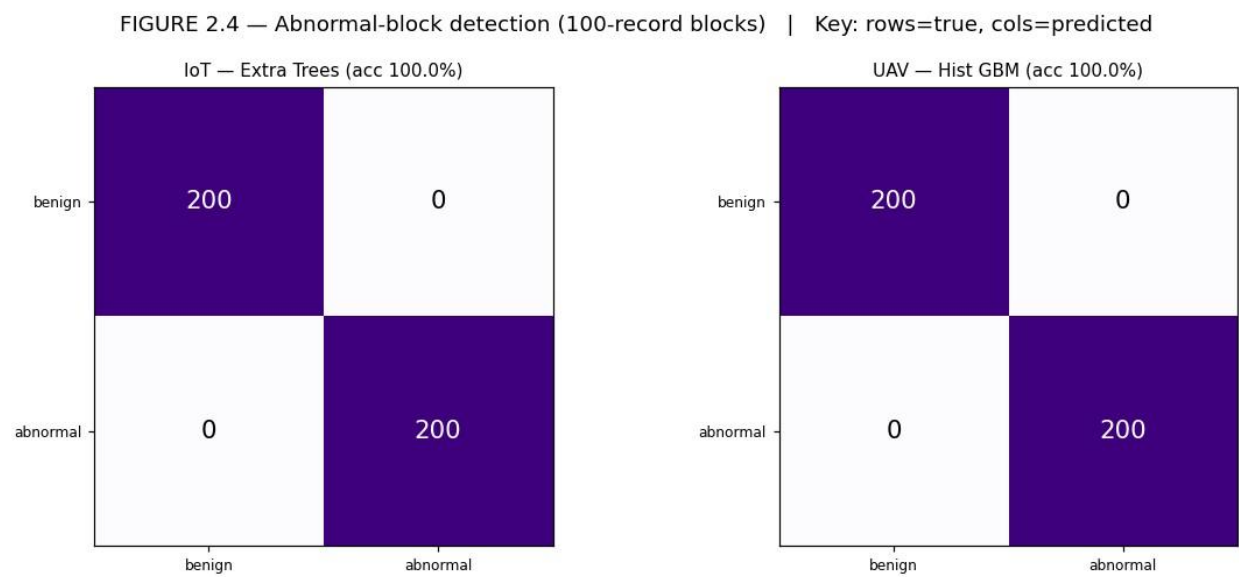
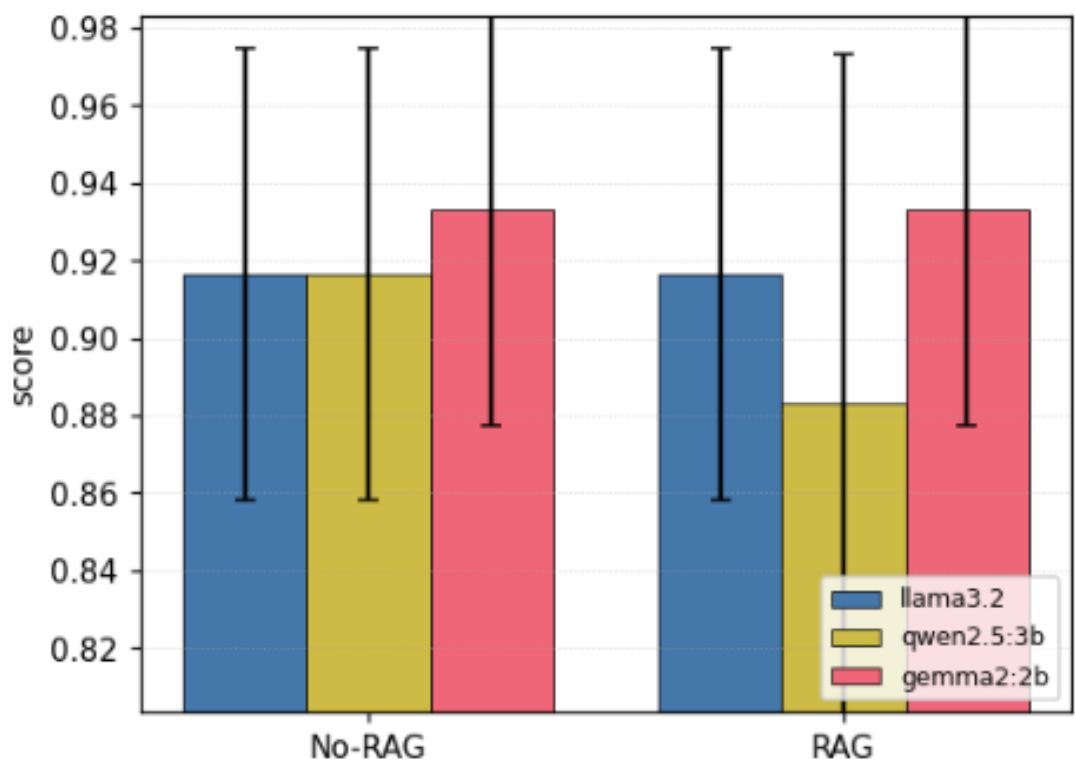
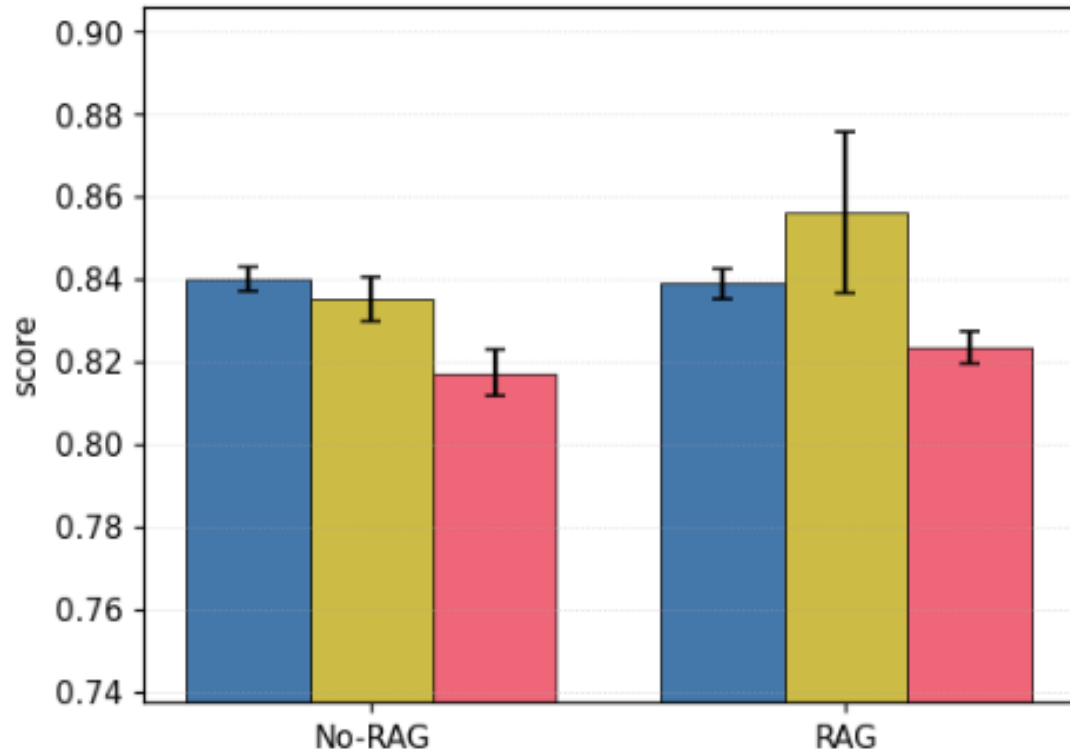


FIGURE 6: LLM DATA (RAG & NO-RAG)

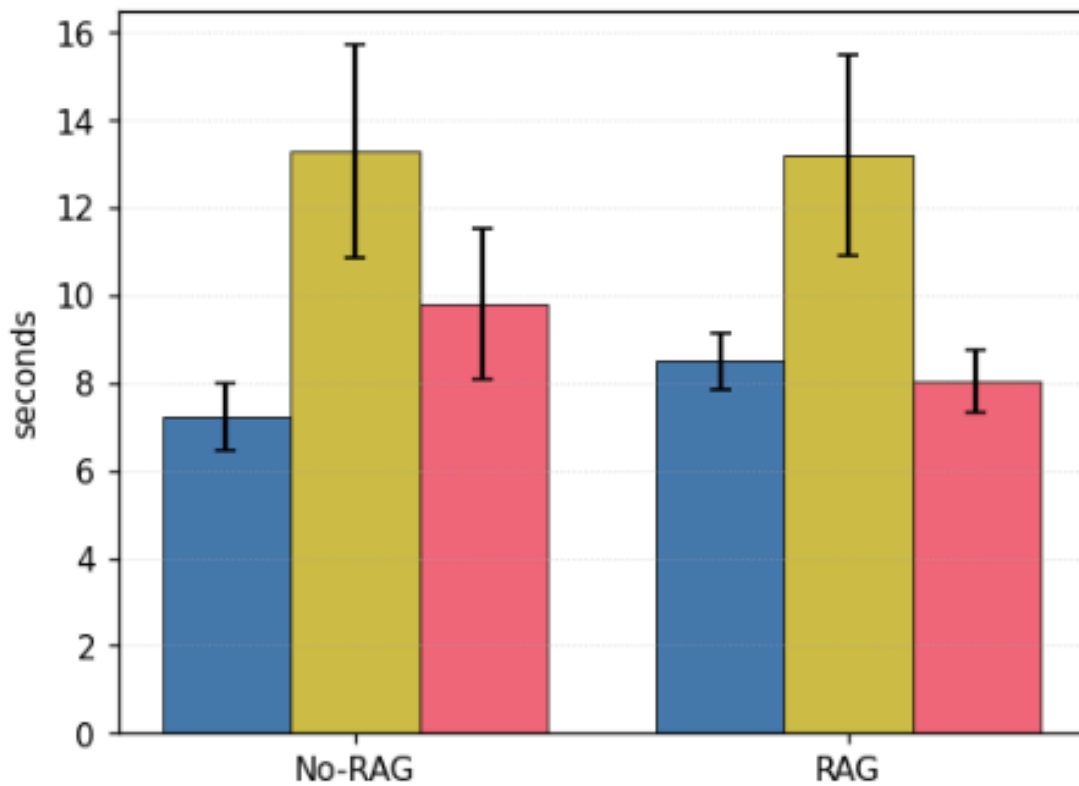
6.1 – MODIFIED CODE CORECTNESS



6.2 – BERTSCORE



6.3 – AVERAGE RESPONSE TIME



CHAPTER 1: INTRODUCTION

The Internet of Things (IoT) and unmanned aerial vehicle (UAV) systems now underpin services on which they rely more heavily than ever on safety, ranging from environmental sensing and industrial control to logistics and aerial inspection. As these systems communicate over shared and often unauthenticated wireless media, a single compromised or misbehaving node can inject falsified telemetry or control frames that propagate through the network and corrupt the decisions that depend on them. This dissertation is a security engineering and data science project: it reviews how existing designs secure such networks, identifies where they fall short, and then designs and empirically evaluates a stronger alternative. The overall aim is to build and measure an explainable framework that combines a blockchain ledger, classical artificial intelligence (AI) detectors and large language model (LLM) assisted interpretation so that misbehaviour in IoT and UAV networks can be detected, resisted at the point of consensus and crucially, explained. The remainder of this chapter sets the scene, states the aim and objectives, outlines the research approach, summarises the contributions and describes the structure of the dissertation.

1.1 Background

The proliferation of IoT sensing and UAV services, often grouped together under the banner of the Internet of Drones (IoD), has extended the attack surface of everyday infrastructure into physical and safety relevant domains [1] [2]. In such deployments trust has traditionally been anchored in a central server or ground station, an arrangement poorly suited to devices that are distributed, mobile and only intermittently connected. Blockchain has therefore been proposed as a decentralised and tamper evident ledger for device data and authorization as once a record is committed it cannot be altered retrospectively and no single party controls admission to the chain [3] [4]. Aggarwal et al. [3] demonstrated this idea for the IoD with an Ethereum based model that provides authentication, authorisation and accountability, while broader surveys have catalogued the consensus mechanisms and security layers that such architectures employ [1] [2].

Two questions, however, are seldom addressed together. The first concerns which consensus mechanism should govern an IoT or UAV ledger. Proof of work, stake based, authority based and Byzantine fault tolerant protocols trade security, latency, communication cost, decentralisation and energy very differently, yet these tradeoffs are usually quoted from general literature rather than measured on the target traffic [1] [2]. The second question concerns what happens once a ledger exists, how are malicious or

abnormal blocks identified and can that identification be justified? In safety critical UAV operation rejecting a frame is effectively an act of censoring data on the ledger, so an operator must be able to audit why a record was flagged. Reviews of blockchain assisted UAV communication note exactly this immaturity of empirically grounded and explainable security pipelines [1], which is the space this project occupies.

Detecting misbehaviour on device traffic is itself an active field. Simple rule-based filters struggle against attacks that use legitimate identifiers and well formed payloads, which has motivated learned detectors trained on labelled traffic. Two open datasets have become common reference points and are used throughout this work. TON_IoT provides telemetry from several IoT sensor types with each record labelled either as normal or as one of nine attack classes [5], while the UAVCAN dataset captures controller area network frames from a Pixhawk UAV testbed under flooding, fuzzy and replay attacks [6]. A recurring weakness in the literature is that such detection is treated as a standalone classification task, decoupled from the ledger that would make its verdicts tamper evident and auditable, so that the security and the trust layers are designed and evaluated in isolation.

A further limitation is opacity. The strongest detectors are typically tree ensembles or neural networks whose decisions are not directly interpretable, which is a serious drawback when a rejected frame carries operational consequences. Model agnostic explainability methods address this by attributing a prediction to its input features: LIME fits a sparse local surrogate around a single instance [7], while Kernel SHAP assigns each feature a Shapley value with additive guarantees that also support a global ranking [8]. Applied to a detector, these methods reveal what actually drives a decision rather than what an analyst assumes drives it and they can therefore inform the very rules that a smart contract enforces.

Even a faithful attribution, however, is difficult for a non-specialist operator to interpret and act upon quickly. Recent work in network security has begun to place an LLM after the explainability layer in order to translate technical attributions into human readable insight and, where the model is capable, to propose mitigations. Chatzimiltis et al. combines LIME and SHAP with an LLM to convert per instance explanations into natural language for non-expert users [10], and a companion study frames LLM assisted, retrieval augmented reasoning for security tasks and compares several generative models on correctness, semantic fidelity and response time [9]. Together these works suggest a route from a flagged and explained attack to an actionable and auditable response, but they target radio access networks rather than blockchain secured IoT and UAV systems and they leave open how such a stage would behave in the domain studied here.

Across this landscape three gaps recur: Consensus choices are rarely measured on domain traffic, AI based misbehaviour detection is treated separately from the ledger, and the resulting decisions remain opaque and hard to act upon. An earlier stage of the present work (excluding the LLM section) began to close the first three of these within a single reproducible framework and was published as a co-authored conference paper (NextGCom 2026) [11], which benchmarked eight consensus algorithms and two domain smart contracts on real IoT and UAV traffic and added explainable AI detection. That paper did not, however, take the final step of turning explanations into justified, human readable mitigations. The problem this dissertation addresses is therefore how to detect misbehaviour in IoT and UAV networks on measured rather than assumed evidence, resist it at the point of consensus, explain each decision and translate that explanation into an actionable and auditable response, all within one coherent and reproducible framework. Solving it matters because it would let designers of next generation IoT and UAV systems choose consensus, detection and explanation mechanisms on the basis of evidence and would give operators a defensible account of why any given record was admitted to (or kept off) the ledger.

1.2 Research Aim and Objectives

The aim of this project is to design and empirically evaluate an explainable blockchain and AI framework for detecting and justifying node misbehaviour in IoT and UAV networks so that system designers can select consensus, detection and explanation mechanisms based on measured evidence.

To achieve this, the following measurable objectives are pursued:

1. To review and critically evaluate the state of the art in blockchain for IoT and UAV security, consensus mechanisms, AI based misbehaviour detection and explainability to establish the research methodology and to identify the gaps this project addresses.
2. To select suitable open and labelled datasets that realistically represent IoT telemetry and UAV controller area network traffic and to prepare them as transaction workloads for the framework.
3. To design and implement a unified framework that maps each dataset record to a ledger transaction and benchmarks eight consensus algorithms and two domain smart contracts on measured security, communication, decentralisation and energy indicators.

4. To integrate ten classical AI detectors for misbehaviour detection at the record and block level and to explain their decisions using LIME and Kernel SHAP.
5. To extend the framework with an LLM assisted interpretation stage that converts explanations into human readable justifications and candidate mitigations and to evaluate it with and without retrieval augmentation across several generative models.
6. To evaluate the complete framework on both datasets and to critically discuss the findings against the aim, the objectives and the reviewed literature.

1.3 Project Approach

The project is conducted as an empirical, experiment led research study supported by a substantial software artefact, rather than as a purely theoretical or purely developmental piece of work. The approach follows a sequence that mirrors the objectives. It begins with a critical literature review, which both establishes the state of the art and justifies the choice of a data driven experimental methodology in preference to a survey only or formal analytical treatment. Two open and labelled datasets are then selected so that the two domains (IoT telemetry and UAV controller area network traffic) can be studied under directly comparable conditions and so that the results can be reproduced by others.

The framework itself is implemented in Python and treats every dataset record as a transaction that passes through a two-layer smart contract before a consensus layer commits it to a ledger, after which an AI layer screens the committed blocks and an explainability layer accounts for each verdict. Because the contract (consensus and detection layers) all operate on the same labelled transaction stream, security and performance are measured end to end rather than assumed, which is central to the contribution. Experiments are controlled and repeatable as they have a fixed block size, a defined adversary fraction and averaging over multiple random seeds are used so that reported differences reflect the mechanisms under study rather than sampling noise. Evaluation is multi-faceted, combining blockchain key performance indicators, classifier metrics such as precision, recall, F1 score and the area under the receiver operating characteristic curve, confusion matrices and for the LLM stage it also includes code correctness, BERTScore and response time. The final LLM assisted stage is run on real, freely available local models (via Ollama) and is compared under retrieval augmented and non-augmented settings.

The study involves no human participants and uses only publicly available and anonymised datasets, so no ethical approval relating to human subjects was required. However, the work does concern offensive traffic and detection tooling and it has been conducted

responsibly by using the datasets solely for defensive evaluation (no attack capability is developed or released) and the emphasis throughout is on detecting and explaining misbehaviour rather than on reproducing it.

1.4 Contributions

The main contributions of this dissertation are:

- A unified and reproducible framework that runs real blockchain consensus and smart contracts directly on real IoT and UAV traffic by mapping every dataset record to a ledger transaction so that the two domains can be compared on a common basis.
- A measured benchmark of eight consensus algorithms and two domain smart contracts across both datasets reported on security, communication, decentralisation and modelled energy indicators rather than on figures quoted from the wider literature.
- An integration of ten classical AI detectors for misbehaviour detection at the record and block level, compared through confusion matrices, receiver operating characteristic analysis and blockchain facing indicators such as the false negative and false positive rates.
- A from first principles LIME and Kernel SHAP explainability layer that produces an actionable finding, namely that the UAV attacks are protocol conformant payload injections rather than timing floods, which explains why deterministic contract rules fail and learned, explainable detection is required.
- An LLM assisted interpretation stage, which is the principal extension beyond the co-authored conference paper [11], that turns each explained detection into a human readable justification and a candidate smart contract policy rule and evaluates this across several local generative models under retrieval augmented and non-augmented settings.

1.5 Structure of the Dissertation

The dissertation is organised as follows. Chapter 2 presents the literature and technology review, critically comparing existing work on blockchain for IoT and UAV security, consensus mechanisms, AI based intrusion and misbehaviour detection, explainability, and the emerging use of large language models as well as positioning the present work against that body of research. Chapter 3 sets out the research approach, reviewing candidate research and development methodologies selecting and justifying the one

adopted, and explaining how the data is accessed and how the study's ethical considerations are addressed. Chapter 4 describes the methodology in detail using two parts covering the system design and the system development which include: the datasets and their preparation as transaction workloads, the two layer smart contracts, the consensus layer, the AI detectors, the explainability methods and the LLM assisted stage, together with the design choices made and their justification, including a subsection on the co-authored conference paper. Chapter 5 tests the framework and presents and critically evaluates the results obtained on both datasets against the aims, the objectives and the reviewed literature. Chapter 6 concludes the dissertation, revisits the objectives, reflects on the limitations and sets out directions for future work.

CHAPTER 2: LITERATURE REVIEW

This chapter reviews the body of research on which the dissertation builds and positions the project against it. It is organised around the layers of the proposed framework so that the reader can see what has been done before for each layer and where it falls short.

Section 2.1 examines blockchain for the Internet of Things and the Internet of Drones.

Section 2.2 reviews the consensus mechanisms that govern such ledgers. Section 2.3 surveys AI based misbehaviour and intrusion detection for IoT and controller area network traffic. Section 2.4 considers explainable artificial intelligence for security and Section 2.5 the emerging use of large language models to interpret and act on detections. Section 2.6 draws these strands together into a critical comparison of representative systems, identifies the gaps that remain and states the research questions and the approach that the rest of the dissertation follows. Section 2.7 summarises the chapter. Throughout, the emphasis is on recent work, with older publications cited only where they define a technique that remains in use.

2.1 Blockchain for IoT and the IoD

Securing IoT and UAV networks matters because these systems increasingly mediate physical processes, so a corrupted record is not merely a data quality problem but a safety one. The dominant response in recent literature is to replace a single trusted server with a blockchain; a decentralised and tamper evident ledger on which committed records cannot be altered retrospectively and admission is not controlled by any one party [4]. For the Internet of Drones specifically, Aggarwal et al. [3] proposed an Ethereum based public blockchain model for secure data dissemination that delivers authentication, authorisation and accountability. It is an influential design and establishes the feasibility of a ledger for drone data but it commits to a single consensus setting and does not consider how alternative mechanisms would perform, nor does it add any learned detection of malicious content once records are on the chain.

Two recent surveys map the wider design space. Jain et al. [2] provides a walkthrough of blockchain based Internet of Drones architectures, cataloguing the consensus and security layers that such systems employ, while Hafeez et al. [1] gives a comprehensive survey of blockchain assisted UAV communication. Both are valuable as orientation and both are current, but by their nature they describe and classify rather than measure as the comparative and quantitative behaviour of the catalogued mechanisms on real drone traffic is left open, and Hafeez et al. [1] explicitly identify the immaturity of empirically grounded and explainable security pipelines as an outstanding problem. Earlier foundational work on lightweight blockchain for IoT, such as Dorri et al. [12], who remove

proof of work and coins to suit constrained devices and demonstrate the idea on a smart home, shares the same limitation from the opposite direction where it fixes one bespoke design, evaluates it only in simulation and neither compares consensus mechanisms nor inspects transaction content for attacks. At an architectural level, Salah et al. [21] reviews the convergence of blockchain and AI and argues that an immutable ledger can make model decisions auditable and trustworthy but the work is forward looking and stops short of a concrete, measured detection pipeline. The recurring pattern in this literature is therefore breadth of design without measured, integrated evaluation, which is the space the present project targets.

2.2 Consensus Mechanisms for Distributed Ledgers

The security and performance of any ledger rests on its consensus mechanism, and the field offers a wide and heterogeneous set. Proof of work, introduced with Bitcoin [4], secures the chain through computational effort but at a well known energy cost. Byzantine fault tolerant protocols such as Practical Byzantine Fault Tolerance [14] instead tolerate malicious proposers through quorum agreement at the price of all to all messaging that scales quadratically with the number of participants. Crash fault tolerant protocols such as Raft [15] are far cheaper to run but assume that nodes fail by stopping rather than by lying, an assumption that matters a great deal in an adversarial setting. Around these poles sit stake based, delegated, authority based and capacity-based variants that trade security, latency, communication cost, decentralisation and energy in different proportions.

Xiao et al. [13] provides a comprehensive and current survey of these distributed consensus protocols, systematically contrasting the security, decentralisation and communication cost of the Byzantine and Nakamoto style families. It is an excellent design reference but it is deliberately qualitative and application agnostic, so the behaviour of these protocols on a particular IoT or UAV workload is not quantified. For aerial systems specifically, Mehta et al. [20] survey blockchain envisioned UAV, communications and UAV stack; again the treatment is conceptual and offers neither an empirical consensus comparison on drone traffic nor a coupled detection layer. The critical observation across this section is consistent with Section 2.1 where consensus tradeoffs in the IoT and UAV literature are almost always quoted from general sources rather than measured on the target traffic, which is precisely what makes an informed choice difficult for a system designer and what the first part of this project sets out to remedy.

2.3 AI Based Misbehaviour and Intrusion Detection

Because attacks on device traffic increasingly use legitimate identifiers and well-formed payloads, deterministic filtering is insufficient and learned detection has become central. Two open and labelled datasets anchor much of the recent work and are used in this project. TON_IoT provides heterogeneous telemetry from several IoT sensor types, each record labelled normal or as one of nine attack classes [5], and the UAVCAN dataset captures controller area network frames from a UAV testbed under flooding, fuzzy and replay attacks [6]. Both are recent and both are designed explicitly for data driven intrusion detection, this makes them appropriate for a study that treats records as ledger transactions.

On the modelling side, Ferrag et al. [16] survey and empirically compare deep learning intrusion detectors across a large collection of datasets and provide a widely used comparative baseline, but the study is confined to centralised detection and considers neither a ledger to protect its decisions nor any means of explaining them. For in vehicle controller area network traffic, which is closely related to the UAVCAN bus studied here, Song et al. [17] detect intrusions with a deep convolutional network over sequences of CAN identifiers and report low missed detection rates on flooding, fuzzy and spoofing attacks therefore the detector is nonetheless a black box that is validated only on ground vehicles and operates in isolation from any ledger. Closest to the present setting, Abdel-Basset et al. [18] combine federated deep learning with blockchain for intrusion detection in smart transportation systems and evaluate on the Car Hacking and TON_IoT datasets, but blockchain there secures the federated training process rather than serving as the data ledger whose consensus is benchmarked, and the resulting detector remains unexplainable. Two critical themes emerge. First, the strongest reported detectors are deep or ensemble models whose gains come at the cost of interpretability. Second (and more importantly for this project) detection is almost universally treated as a standalone classification problem, decoupled from the consensus layer that would make its verdicts tamper evident, so that the two halves of a trustworthy pipeline are rarely designed or measured together.

2.4 Explainable AI for Security

The opacity noted above is a serious obstacle in a security setting, where rejecting a record has operational consequences and an analyst must be able to justify the decision. Two model agnostic methods dominate. LIME fits a sparse local surrogate around an individual prediction to approximate a complex model near that instance [7], and Kernel SHAP attributes to each feature its Shapley value, with additivity guarantees that also allow the local attributions to be averaged into a global importance ranking [8]. The two are

complementary as LIME is fast and intuitive but its explanations can be unstable while SHAP is more principled but more expensive, therefore an agreement between them is a good sign and raises confidence in the features identified.

Their application to intrusion detection is recent and growing. Wang et al. [19] were among the first to apply SHAP to intrusion detection, combining local and global attributions so that analysts can understand why a classifier flags a record. The work is a useful demonstration but it is limited to a single conventional network dataset, it's not coupled to a blockchain, and it isn't extended to IoT or UAV domains or cross checked against a second explainer. The critical point for this dissertation is that explainability in security has so far been used mainly to interpret a detector after the fact and rarely to feed back into the very rules a system enforces. In a blockchain setting, where a smart contract encodes those rules, a faithful attribution is not only an audit aid but a potential design input, and this project uses it in exactly that way, which distinguishes it from prior XAI for intrusion detection studies.

2.5 LLMs for Security Interpretation and Mitigation

Even a faithful feature attribution is difficult for a non-specialist operator to interpret and act upon quickly, and the most recent strand of relevant work places a large language model after the explainability layer to close this gap. Chatzimiltis et al. [10] propose an explainable framework in which LIME and SHAP explanations of an anomaly detector are passed to an LLM that translates the technical attributions into natural language for non-expert users, evaluated in part with BERTScore [22], a metric that scores generated text by its semantic similarity to a human reference using contextual embeddings. A companion study by the same group [9] frames LLM assisted, retrieval augmented reasoning for security compliance and compares several generative models (with and without retrieval augmentation, i.e. RAG and no-RAG) on measured criteria including modified code correctness, semantic fidelity and response time, reporting that retrieval mainly improves the grounding and quality of the generated output at the cost of higher latency.

These works are the closest methodological antecedents of the final stage of this project and they are very recent, which is a strength. However their limitation from the present perspective, is one of domain and integration both are targeting radio access and 5G networks rather than blockchain secured IoT and UAV systems, and neither couples the LLM stage to a ledger whose consensus and smart contracts have themselves been measured. It is therefore an open question whether an LLM assisted interpretation stage, and in particular retrieval augmentation, provides comparable benefits when the underlying detections come from the heterogeneous IoT and CAN traffic studied here and when the target output is a candidate smart contract rule rather than a configuration fix.

This project adapts the evaluation methodology of [9] and the explanation to language pipeline of [10] to answer that question in a new domain.

2.6 Critical Comparison and Research Gaps

The table I have added below compares representative systems from the preceding sections across the capabilities that a trustworthy IoT and UAV security pipeline would ideally combine. The comparison is deliberately capability oriented rather than performance oriented because the works differ in scope and datasets and a like for like accuracy comparison would be misleading, what the table shows instead is which parts of the problem each work addresses.

Work	Ledger / blockchain	Consensus measured on data	Learned detection	Explainability (XAI)	LLM interpretation	Real IoT / UAV traffic
Aggarwal et al. [3]	Yes	No	No	No	No	Yes
Jain et al. [2]	Yes (survey)	Partial	No	No	No	No
Hafeez et al. [1]	Yes (survey)	Partial	No	No	No	No
Dorri et al. [12]	Yes	No	No	No	No	No
Xiao et al. [13]	Yes (survey)	Partial	No	No	No	No
Ferrag et al. [16]	No	No	Yes	No	No	Yes
Song et al. [17]	No	No	Yes	No	No	Yes (CAN)
Abdel-Basset et al. [18]	Yes (for training)	No	Yes	No	No	Yes

Work	Ledger / blockchain	Consensus measured on data	Learned detection	Explainability (XAI)	LLM interpretation	Real IoT / UAV traffic
Wang et al. [19]	No	No	Yes	Yes	No	Partial (NSL- KDD)
Mehta et al. [20]	Yes (survey)	Partial	No	No	No	No
Chatzimiltis et al. [9], [10]	No	No	Yes	Yes	Yes	Yes (RAN / 5G)
This project	Yes	Yes	Yes	Yes	Yes	Yes

The table makes the gap concrete. Blockchain focused work secures the ledger but does not measure consensus on domain traffic and adds no learned or explainable detection. Detection focused work produces strong classifiers but decouples them from any ledger and (with the exception of Wang et al. [19]) leaves them opaque. The XAI to LLM work is the most complete but lives in a different domain and is not tied to a measured ledger. No prior system combines a measured consensus benchmark, ledger coupled and explainable AI detection, and an LLM assisted interpretation stage on real IoT and UAV traffic. Three gaps therefore recur throughout the reviewed literature, and they are that the consensus choices are seldom measured on domain data, the AI based detection is treated separately from the ledger and that the resulting decisions are opaque and hard to act upon.

From these gaps the dissertation derives the following research questions, which the remaining chapters answer and which the evaluation revisits explicitly:

- **RQ1:** Does the choice of consensus mechanism materially affect the security of an IoT or UAV ledger when measured on real attack traffic, and what is the communication and energy cost?
- **RQ2:** Can deterministic smart contract rules alone stop realistic IoT and UAV attacks, or is a learned detector required?

- **RQ3:** Among a range of classical detectors, which best balance security (measured as missed attacks) against availability (measured as falsely rejected records) at both the record and block level?
- **RQ4:** What features actually drive the detectors' decisions, and does explaining them overturn intuitive assumptions about the nature of the attacks?
- **RQ5:** Can an LLM assisted stage convert these explanations into accurate and human readable justifications and candidate mitigations, and does retrieval augmentation improve the result?

The review also justifies the approach adopted in preference to the alternatives. A survey only project would reproduce the descriptive limitation identified above as a purely theoretical treatment could not answer questions that are fundamentally empirical, such as how much attack traffic a given consensus mechanism admits. The gaps are therefore best addressed by an experiment led study built on a single, reproducible software framework that maps real records to transactions and measures every layer end to end, extended with the LLM assisted stage that the most recent literature makes possible but has not yet applied to this domain. This is the approach developed in the chapters that follow.

2.7 Summary

This chapter has critically reviewed the literature across the five layers of the proposed framework and compared representative systems in the table in section 2.6. Blockchain research for the IoT and IoD secures the ledger but rarely measures consensus on domain traffic or adds explainable detection, consensus surveys are thorough but qualitative, AI based detection produces strong but opaque classifiers that are decoupled from the ledger, explainability has been applied to detection mainly as an after the fact audit rather than as a design input and the promising XAI to LLM pipelines target radio access networks rather than blockchain secured IoT and UAV systems. The synthesis identified three persistent gaps, translated them into five research questions and justified an experiment led and reproducible framework as the appropriate response.

The next chapter turns from what has been done to how this project approaches the problem. Chapter 3 presents the research approach by arguing that the questions raised here are empirical rather than descriptive, by reviewing candidate research and development methodologies, by selecting and justifying an iterative and experiment led methodology, and by explaining how the two datasets are accessed and how the study addresses its ethical and responsible research considerations. This provides the bridge to Chapter 4, in which that methodology is realised in detail as the design and development of

the framework, and ultimately to Chapter 5, in which the framework is tested and its results evaluated against the research questions set out above.

CHAPTER 3: RESEARCH APPROACH

This chapter explains how the project is conducted and why that way was chosen, forming a bridge between the gaps identified in Chapter 2 and the detailed design presented in Chapter 4. Section 3.1 reviews candidate research and software development methodologies then selects one and justifies the choice against the nature of the project and the findings of the literature review. Section 3.2 explains concretely how the chosen methodology is applied across the layers of the framework and how the research questions map onto its iterations. Section 3.3 sets out how the data are accessed and prepared, and Section 3.4 addresses the ethical and responsible research considerations that apply. Section 3.5 summarises the chapter and introduces the methodology. By the end of this chapter the reader should understand the shape of the work before meeting its detail.

3.1 Choice of Research Methodology

The literature review established that the outstanding questions in this area are empirical rather than descriptive or purely theoretical. Whether a given consensus mechanism admits attack traffic, whether deterministic rules suffice, which detector best balances security against availability, and whether retrieval augmentation helps an LLM stage are all questions that can only be answered by building a system and measuring it. This framing rules out a survey only project which would reproduce the descriptive limitation criticised in Chapter 2, and it rules out formal analytical treatment, which could not quantify behaviour on real traffic. The project is therefore an experiment led research study supported by a substantial software artefact, and the methodology must serve both the research and the engineering sides of that description.

Two methodology families were considered. On the software engineering side, the classical options are the Waterfall model in which requirements, design, implementation, testing and evaluation proceed in a single linear sequence, and Agile/iterative models in which the system is grown in short cycles that each deliver and evaluate a working increment. Waterfall is attractive for its clear documentation and fixed scope, which suit reproducibility, but it assumes that requirements are well understood in advance and it defers evaluation to the end, which is a poor fit for a project whose very purpose is to discover empirical facts that then shape later decisions. An early result of this project illustrates the point: the finding that the UAV attacks are protocol conformant payload injections rather than timing floods (obtained only once the detectors were built and explained) changed how later components were designed and justified. A method that could not accommodate that kind of feedback would have been unsuitable.

On the research side, the project aligns naturally with an established data science process, exemplified by methodologies such as the Cross Industry Standard Process for Data Mining (CRISP-DM) [23], whose phases move from understanding the problem and the data, through preparation and modelling, then to evaluation and deployment, with explicit loops back to earlier phases as understanding improves. The methodology adopted for this dissertation is an iterative and experiment led process that combines the two, it takes the phased data science structure of CRISP-DM as its backbone and executes each phase in Agile style increments so that each layer of the framework is designed, implemented, measured and then critically reviewed before the next is added so that findings from one increment inform the next. This choice is justified on three grounds: It matches the empirical nature of the questions since every increment ends in measurement, it accommodates the feedback that the literature review predicted would arise since findings propagate forward, and it supports the reproducibility because each increment is documented in a singular and self contained computational notebook with fixed configuration and seeds.

3.2 Application of the Methodology to This Project

In practice the methodology is realised as a sequence of increments that correspond to the parts of the implemented framework and to the research questions of Chapter 2. The first increment establishes problem and data understanding where the two datasets are studied and each record is framed as a ledger transaction so that the two domains can be treated on a common basis. The second increment implements and measures the consensus layer and the two smart contracts, answering RQ1 and RQ2 on real traffic. The third increment adds the ten AI detectors and evaluates them at the record and block level, answering RQ3. The fourth increment builds the LIME and Kernel SHAP explainability layer and interprets the strongest detectors, answering RQ4, and it is here that the feedback from measurement to design is most visible. The fifth increment adds the LLM assisted interpretation stage and evaluates it with and without retrieval augmentation, answering RQ5. A final increment evaluates the complete framework end to end and revisits the objectives, which corresponds to the evaluation phase of the data science process and to Chapter 6 of this report.

Each increment follows the same internal cycle. A component is designed with reference to the literature reviewed in Chapter 2, implemented in Python, exercised on both datasets under controlled conditions, and then critically reviewed with the outcome recorded and any consequence for later increments noted. Controlled conditions here mean a fixed block size, a defined adversary fraction and averaging over several random seeds so that reported differences reflect the mechanisms under study rather than sampling variation.

The whole framework is expressed as a single ordered notebook (Jupyter notebook), which means that a later increment can reuse the fitted models and intermediate results of an earlier one without recomputation, and that the entire study can be reproduced by running the notebook end to end. This organisation directly serves the documentation and reproducibility of the project.

3.3 Data Access and Preparation

The project is a secondary data study as it uses two openly published and labelled datasets rather than collecting new data, which removes any dependence on bespoke data collection and makes the results directly reproducible by others. The TON_IoT telemetry dataset [5] provides labelled readings from several IoT sensor types and is distributed openly for research, and the UAVCAN intrusion dataset [6] provides labelled controller area network frames captured from a UAV testbed and is published through IEEE DataPort. Both were obtained from their public sources and used under the terms on which they were released.

Preparation follows the data understanding and data preparation phases of the adopted process. Balanced transaction streams of equal size were drawn from each dataset so that the two domains are directly comparable, categorical and numerical features were engineered consistently, and for the UAV data two timing features (the inter frame gap and the per identifier gap) are derived in temporal order before sampling so that no ordering information is lost. The datasets are treated purely as workloads for the framework, the labels are used only as ground truth for evaluation and, in one controlled experiment, as an idealised oracle against which realistic detection is compared. The precise engineering steps, feature sets and sampling sizes are design decisions and are documented in Chapter 4.

3.4 Ethical and Responsible Research Considerations

The project involves no human participants and processes no personal data. The two datasets contain machine generated telemetry and network frames rather than information about identifiable individuals, and they are published specifically for security research so no ethical approval relating to human subjects was required and there are no data protection obligations concerning participant data to discharge. Where the raw captures contain incidental network identifiers such as addresses, they are used only as model features and are not linked to any person.

Although no human subjects are involved, the work concerns offensive traffic and detection tooling, and it has been conducted responsibly in the spirit of ethical security research. The datasets are used solely for defensive evaluation as the project develops and

releases no attack capability and its emphasis throughout is on detecting, explaining and mitigating misbehaviour rather than on reproducing it. The LLM assisted stage is run on freely available local models (Ollama) with no transmission of data to third party services, which avoids any incidental data sharing during experimentation. These measures ensure that the research is both compliant and responsible.

3.5 Summary

This chapter has set out the research approach that governs this dissertation. It argued that the questions raised by the literature review are empirical and therefore that an experiment led study built on a reproducible software artefact is the appropriate response rather than a survey or a purely theoretical treatment. It selected an iterative, experiment led methodology that combines the phased structure of an established data science process with Agile style increments, justified that choice against the nature of the project and the feedback that measurement was expected to produce, and explained how the methodology is realised as a sequence of increments mapped onto the research questions. It also described how the two open datasets were accessed then prepared and confirmed that the work raises no human subject ethical concerns while being conducted responsibly with respect to its offensive material.

With the approach established, the next chapter provides the detail. Chapter 4 presents the methodology and design of the framework increment by increment by describing the datasets and their preparation as transaction workloads, the two layer smart contracts, the eight consensus algorithms, the ten AI detectors, the LIME and Kernel SHAP explainability layer and the LLM assisted interpretation stage, justifying each design choice with reference to the literature reviewed in Chapter 2. It also includes a dedicated subsection on my co-authored conference paper that reported the earlier stage of this work.

CHAPTER 4: METHODOLOGY

This chapter presents the design of the proposed framework and the steps taken to develop it. It is organised into two main sections that reflect the distinction between analysis and design on the one hand and documented implementation on the other. Section 4.1 covers the system design including the functional and nonfunctional requirements, the overall architecture, the threat model, and the design of each layer of the framework, with every significant choice justified against the gaps and research questions established in Chapter 2. Section 4.2 covers the system development which includes the environment and tools, and the increment by increment implementation that realises the iterative experiment led methodology adopted in Chapter 3, together with the measures taken to make the work reproducible. Section 4.3 is a dedicated section on my co-authored conference paper that reported the earlier stage of this work. Throughout, the design is described in terms of what was built and why, while the development is described in terms of how it was built and with which parameters so that the two sections complement rather than repeat one another.

4.1 System Design

4.1.1 Requirements Analysis

The requirements follow directly from the aim and the research questions. The functional requirements are that the framework shall: map each record of a labelled IoT or UAV dataset to a ledger transaction, validate every transaction through a two layer smart contract before it is committed, order and commit transactions using a range of consensus mechanisms so that their behaviour can be compared, detect misbehaviour among committed records using trained classifiers at both the record and the block level, explain each detector's decisions in terms of the input features and translate a flagged and explained detection into a human readable justification and a candidate mitigation rule using a large language model. The nonfunctional requirements are that the framework shall be measurable in that every reported quantity is counted or timed rather than assumed, comparable in that the two domains are studied under equivalent conditions, controlled in that results are averaged over repeated runs with fixed configuration so that differences are attributable to the mechanisms under study, and reproducible in that the entire study can be rerun from openly available data. These nonfunctional requirements are a direct response to the central criticism made in Chapter 2, namely that prior work quotes consensus behaviour rather than measuring it and evaluates detection in isolation from the ledger.

4.1.2 Architecture and Threat Model

The framework is a single pipeline in which a dataset record becomes a transaction and that transaction passes through four stages before its verdict is explained and acted upon. First, a smart contract gateway validates the transaction. Second, a consensus layer orders validated transactions into blocks and commits them to a distributed ledger. Third, an AI misbehaviour detection layer screens the committed records and the blocks that contain them. Fourth, an explainability layer accounts for each detector decision, and a large language model stage converts that account into a justification and a candidate policy rule, whose attributions can in turn inform the contract. Because the contract, consensus and detection layers all operate on the same labelled transaction stream, security and performance are measured end to end. A schematic of this pipeline was presented in the co-authored conference paper [11] and the present chapter describes each stage in turn.

The single modelling assumption is the adversary. The design considers a network of block producers of which a fraction are Byzantine meaning that honest producers execute the contract faithfully and drop rejected transactions, whereas Byzantine producers attempt to commit their batch wholesale so that any attack transactions they carry bypass validation. This adversary fraction is the only quantity that is assumed rather than taken from the data but the attack workload itself is drawn entirely from the datasets. A block groups a fixed number of transactions, and for each algorithm the framework records the number of protocol messages, the distribution of block production across producers and whether each committed transaction is benign or an attack (as the datasets are labelled) from which the security indicators are derived.

4.1.3 Data Design: Datasets as Transaction Workloads

Two open and labelled datasets are used as workloads, chosen because they are recent, purpose built for data driven intrusion detection, and representative of the two target domains. The TON_IoT telemetry dataset [5] supplies heterogeneous readings from seven IoT sensor types, and the UAVCAN dataset [6] supplies controller area network frames captured from a UAV testbed under flooding, fuzzy and replay attacks. Each record is mapped to a transaction consisting of a device or identifier, a payload and a ground truth label. For the UAV data the payload comprises the CAN identifier, the data length code, eight payload bytes and two timing features, the inter frame gap and the per identifier gap which are all derived in temporal order before any sampling so that no ordering information is lost. Balanced streams of equal size are drawn from each dataset so that the two domains are directly comparable. The labels are treated strictly as ground truth for

evaluation and as an idealised oracle, and are never presented to the detectors as an input feature.

4.1.4 Smart Contract Design

Each domain has a two layer contract, a design that separates what can be checked deterministically from what must be learned. The deterministic rule layer is bootstrapped from normal traffic only and enforces three conditions:

- a device or identifier registry
- payload well formedness
- a per identifier rate limit.

For the IoT contract the rule layer checks that the sensor is on the registry. For the UAV contract it additionally checks that the data length code is legal (between one and eight) and that the per identifier gap is not shorter than a minimum, which acts as a flooding guard. Above this sits a pluggable policy layer that supplies an anomaly decision. A transaction is admitted if and only if it passes the rule layer and is not flagged by the policy, which can be written as:

$\text{ADMIT}(t) = \text{RULE}(t) \wedge \neg P(t)$, where $\text{RULE}(t) = \text{REG}(t) \wedge \text{FMT}(t) \wedge \text{RATE}(t)$.

The policy P is deliberately interchangeable. In the first part of the study it is an oracle that returns the ground truth label, which yields an upper bound on achievable security, and it is then replaced by the trained detector of Section 4.1.6 to quantify the cost of realistic detection. This design lets the framework separate the contribution of the deterministic rules from that of the learned policy (precisely the separation that Chapter 2 found to be missing in prior work) where detection and ledger are not co designed.

4.1.5 Consensus Layer Design

Eight consensus algorithms are implemented (with their genuine mechanics rather than as abstract stand ins) so that the security and cost differences discussed in Section 2.2 are exercised on real traffic. They span three families each comprising of:

1- The economic family:

- Proof of Work which secures the chain through real hashing effort.
- Proof of Stake in which a stake weighted proposer is subject to slashing when a malicious block is detected.
- Delegated Proof of Stake in which a small elected set of producers is chosen by stake weighted vote.
- Proof of Capacity, which weights producers by committed storage.

2- The leader based family:

- Proof of Authority in which vetted validators operate under a lower adversary fraction and a high audit probability.
- Raft, which is a crash fault tolerant protocol where a lying leader can replicate a corrupt log because the protocol assumes nodes fail by stopping rather than by cheating.

3- The Byzantine fault tolerant family:

- Practical Byzantine Fault Tolerance where a three phase agreement with an all to all message pattern and a two thirds quorum that overrules a malicious proposer.
- Delegated Byzantine Fault Tolerance, a committee-based variant.

The principal security indicator is the integrity I , the fraction of attack transactions kept off the ledger,

$$I = N_{\text{rejected}}^{\text{mal}} / (N_{\text{rejected}}^{\text{mal}} + N_{\text{committed}}^{\text{mal}}),$$

and decentralisation is measured through the Gini coefficient of the per producer block counts, reported as $1 - G$ so that a higher value is more decentralised. The framework also records the message count per block, the block time and a modelled relative energy that counts hashing effort plus messaging. Selecting eight protocols across three families, rather than one or two, is a deliberate response to RQ1.

4.1.6 AI Misbehaviour Detection Design

Because Chapter 2 showed that realistic attacks use legitimate identifiers and well formed payloads, the policy layer is realised by learned detectors rather than by further handwritten rules. Ten classical detectors are used, chosen to cover the main families of the classical toolkit so that the comparison is representative rather than tuned to a single favoured model, and they include linear and probabilistic models, instance based and tree based models, and boosting ensembles. Each is trained per dataset on observable features only, with the label used solely as the target. Detectors are assessed both by standard classification metrics (including precision, recall, the mean F1 score and the area under the receiver operating characteristic (ROC) curve) and more importantly for a security setting by two error costs that the ledger makes meaningful, which are the false negative rate (which counts attacks admitted to the ledger and is therefore a security failure) and the false positive rate (which counts valid records rejected and is therefore an availability cost). Detection is also lifted from the record to the block because the block is the unit that consensus commits. A block is flagged as abnormal when at least half of its records are predicted malicious,

$$\text{flag}(B) = 1 \text{ if } (1/|B|) \sum_{t \in B} \text{pred}(t) \geq 0.5, \text{ else } 0,$$

which allows an imperfect record level detector to identify an attack heavy block reliably since the evidence compounds over the block. Reporting both the record and block level as well as both the classification and the blockchain facing error costs directly answers RQ3, which asks which detectors best balance security against availability at each level.

4.1.7 Explainability Design

To satisfy the requirement that decisions be auditable, the framework explains its detectors with two complementary model agnostic methods implemented from first principles so that no functionality is hidden inside an external explainer and so that the mechanics are fully documented. LIME fits a sparse local surrogate around an instance by weighting perturbations according to their proximity and penalising complexity, and Kernel SHAP attributes to each feature 'i' its Shapley value

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} (f(S \cup \{i\}) - f(S)),$$

which is additive and can therefore be averaged in absolute value over many instances to yield a global feature ranking. The two methods are used together, with SHAP treated as primary and LIME as corroboration, and their top feature agreement is measured because, as noted in Chapter 2, LIME explanations can be unstable and that agreement between independent explainers raises confidence. This layer is designed to do real work rather than to decorate the results by revealing which features actually drive detection, therefore it answers RQ4 and overturns an intuitive assumption about the UAV attacks that in turn justifies the learned detection layer (as the results will show in Chapter 5).

4.1.8 LLM Assisted Interpretation Design

The final stage addresses the observation in Section 2.5 that even a faithful attribution is hard for a non-specialist to act upon quickly. It adapts the explanation to language pipeline of Chatzimiltis et al. [10] and the retrieval augmented evaluation methodology of the companion study [9] to the present domain. For each flagged attack the per instance SHAP and LIME attributions, the short global feature ranking, the simple per feature statistics for normal and attack traffic, and the detector's verdict are assembled into a structured prompt following the schema of the paper[10]. A generative model then returns a plain language explanation of why the record is malicious and a candidate smart contract policy rule (expressed as a small function) that would block the pattern. The stage is evaluated under two retrieval settings, with and without retrieval augmented generation (RAG and no-RAG) [25], where retrieval draws the most relevant entries from a small domain knowledge base. Three metrics mirror those of the companion study [9]:

- A code correctness score for the generated rule.
- BERTScore [22] for the semantic fidelity of the explanation against a domain reference.

- Response time per call.

Adapting an established methodology to a new domain, rather than inventing an ad hoc evaluation, this allows RQ5 to be answered in terms that are comparable to the source literature.

4.2 System Development

4.2.1 Development Environment and Tools

The framework is implemented in Python and expressed as a single ordered computational notebook so that a later stage can reuse the fitted models and intermediate results of an earlier one and so that the whole study runs as one end to end. The consensus and contract layer as well as the explainers and the block level test are written from first principles using only the numerical libraries NumPy and pandas, while the ten detectors and the preprocessing pipeline use scikit-learn [24]. The large language model stage runs three open models locally through the Ollama runtime (which requires no paid interface and keeps all data on the machine) and it uses scikit-learn for retrieval and the bert-score library for evaluation. The development followed the iterative and experiment led methodology of Chapter 3 where each of the following increments designs, implements, measures and then reviews a layer of the framework before the next is added, and each is realised as one or more cells of the notebook with fixed configuration constants declared once at the top.

4.2.2 Increment One: Data Ingestion and Transaction Mapping

The first increment establishes the common workload. Balanced streams of forty-two thousand transactions are drawn from each dataset, formed from six thousand records for each of the seven IoT sensor types and four thousand two hundred records for each of the ten UAV capture files. Each record is parsed into a transaction with a device or identifier, a payload dictionary and a Boolean attack label, and the two contract registries are bootstrapped from the normal traffic only. The measured attack prevalence of the resulting streams is 59.7% for the IoT stream and 36.6% for the UAV stream, and these figures are reported rather than balanced away so that the evaluation reflects the datasets as published.

4.2.3 Increment Two: Consensus and Smart Contract Implementation

The second increment implements the two contracts and the eight consensus algorithms and runs them on both streams. A block size of one hundred transactions is used throughout, results are averaged over three random seeds, and the adversary fraction is set to 20% (reduced to 5% for the vetted Proof of Authority validators). Modelled constants that

are documented rather than measured, namely the audit and crash probabilities and the unit hashing and message costs, are declared once so that they are transparent and adjustable. Honest producers run the contract and drop rejected transactions, while Byzantine producers commit their batch wholesale, and the per run counters are then consolidated into the integrity, communication, decentralisation, latency and energy indicators as defined in Section 4.1.5. The resulting key performance indicators are reported in Chapter 5 in Table 1 and Figure 1.

4.2.4 Increment Three: Feature Engineering and Detector Training

The third increment prepares features and trains the detectors. Transactions are converted to a numeric feature frame with the labels excluded, after which a column transformer imputes and standardises the numerical features and one hot encodes the categorical features with unknown categories ignored at transform time so that the pipeline is robust to unseen values. Each dataset is split seventy to thirty with stratification and a fixed random seed, which yields twelve thousand six hundred test transactions per dataset and, after encoding, thirty-eight features for the IoT stream. The ten detectors are then fitted on the training partition, and they are:

- logistic regression with a raised iteration limit
- linear discriminant analysis
- Gaussian naive Bayes
- k nearest neighbours with seven neighbours
- a decision tree
- random forest
- extra trees ensembles of one hundred and twenty trees each
- an AdaBoost ensemble
- histogram gradient boosting
- a linear support vector classifier

all with fixed random seeds where applicable. The record level metrics appear in Chapter 5 in Table 2, Table 3 and Figure 3, the receiver operating characteristic analysis in Figure 4 and Table 4, and the block level test (which draws two hundred blocks with adversarial blocks composed of 70% attack records and flags a block at the half predicted malicious threshold) in Figure 5.

4.2.5 Increment Four: From First Principles LIME and Kernel SHAP

The fourth increment implements the two explainers directly. For LIME, an instance is perturbed by sampling from the training mean and standard deviation, each perturbation is weighted by an exponential kernel on its standardised distance to the instance, its features are binarised by quartile membership and a ridge surrogate is fitted whose coefficients are

the local importances. For Kernel SHAP, feature coalitions are sampled and weighted by the Shapley kernel, masked inputs are formed by combining the instance with a background sample drawn from the training data, and a weighted linear regression without intercept is fitted to the model output so that its coefficients approximate the Shapley values. The strongest detector on each dataset is explained, per instance attributions are averaged in absolute value into the global rankings (shown in Chapter 5 in Figure 2), and the top feature agreement between the two methods is computed. This increment produced the study's key interpretive finding (which then fed back into the justification of the detection layer) which is an instance of the design to measurement feedback that motivated the iterative methodology in Chapter 3.

4.2.6 Increment Five: LLM Assisted Stage Implementation

The fifth increment builds the interpretation stage. A domain knowledge base of sixteen short passages (covering the attack types, the contract layers, the feature meanings and candidate mitigations) is indexed with a term frequency inverse document frequency vectoriser, and retrieval returns the three most relevant passages for a query built from an instance's leading features. For each of six sampled attacks per dataset, the structured prompt of Section 4.1.8 is constructed in a without retrieval and a with retrieval variant, and three local models (llama3.2, qwen2.5:3b and gemma2:2b) are queried at a fixed low temperature. The generated rule is scored by a five-part correctness rubric that checks valid Python syntax, the correct function signature, a reference to the leading feature, the presence of a threshold or condition, and the issuing of a decision. The explanation is scored with BERTScore using a baseline free RoBERTa large model against a domain reference, and the response time is measured after a one off warm up call per model so that the timing excludes model loading. The outcomes for both retrieval settings across the three models are reported in Chapter 5 in Table 5 and Figure 6.

4.2.7 Reproducibility and Validation

Reproducibility was treated as a first class requirement throughout the project, this was achieved by having:

- All configurations declared once as named constants.
- Datasets that are openly available.
- Every random process seeded.
- Results averaged over repeated runs.
- The whole framework in a single notebook that can be executed end to end.

The framework was validated at each increment against internal consistency checks (for example confirming that the deterministic rule layer alone stops almost no attacks, which is the expected consequence of attacks that use registered identifiers and legal payloads)

and that the abnormal block test reaches full accuracy, which is the expected consequence of evidence compounding over a hundred record block. Where a component was mocked for offline development, such as the language model when no local server is present, the framework records the provenance of every result and clearly labels any non-genuine output so that only real measurements are reported.

4.3 NextGCom 2026 Conference Paper

The consensus, smart contract, detection and explainability layers described above, corresponding to the first two parts of the implementation in the notebook, was developed by me and published as a co-authored conference paper to NextGCom 2026 [11], on which I am the lead author. That paper presented the unified framework, the measured benchmark of the eight consensus algorithms and two contracts, the ten detectors with their record and block level evaluation, and the from first principles LIME and Kernel SHAP layer, and it reported the key finding that the UAV attacks are protocol conformant payload injections. The dissertation uses and documents that validated core in full, and its principal original extension is the large language model assisted interpretation stage of Section 4.1.8, which was not part of the conference paper. The relationship is therefore one of a peer reviewed foundation which lends external validation to the first two parts of the framework, and a new stage built upon it which is developed and evaluated for the first time in this dissertation. This division is revisited in the evaluation, where the contribution of the new stage is assessed on its own terms.

4.4 Summary

This chapter has presented the design and development of the framework. The design section derived functional and nonfunctional requirements from the aim, set out an end to end architecture and a single adversary assumption and justified the design of each layer, the datasets as workloads, the two layer contracts, the eight consensus algorithms, the ten detectors at the record and block level, the two explainers and the language model stage, against the gaps and research questions of Chapter 2. The development section described the Python implementation as a sequence of measured increments that realise the iterative methodology of Chapter 3, gave the parameters needed to reproduce each increment, and explained how the earlier two parts were validated through my co-authored conference paper while clarifying that the language model stage is exclusive to the dissertation.

The next chapter presents and critically discusses the results that this framework produces. Chapter 5 reports the measured consensus indicators, the record and block level detection performance, the explanations and the key interpretive finding they yield,

and the behaviour of the language model stage under the two retrieval settings, drawing on Figures 1 to 6 and Tables 1 to 5. It also evaluates each outcome against the aim, the objectives and the reviewed literature.

CHAPTER 5: TESTING, RESULTS AND EVALUATION

This chapter tests the framework, presents the results it produces, and evaluates them critically. Because the project is an experiment led research study rather than an interactive application, testing is understood here as the verification and validation of the experimental framework confirming that the implementation faithfully realises the design of Chapter 4 and that it functions correctly end to end, rather than user acceptance testing of a piece of software. Section 5.1 sets out this testing and verification approach and assesses the framework against the requirements of Section 4.1.1. Sections 5.2 to 5.5 then present and discuss the results in the order of the research questions from Chapter 2 (the consensus and smart contract behaviour, then the record and block level detection performance, then the explanations and the key interpretive finding they yield, and finally the behaviour of the language model stage). Section 5.6 conducts a thorough comparison with existing research and highlights the novel aspects of the work. Section 5.7 evaluates the results critically and states their limitations, and Section 5.8 evaluates the project objectives. Section 5.9 then summarises the chapter. All figures and tables referred to are collected in the List of Figures and List of Tables sections at the start of this paper and are cited by their number when referred to below.

5.1 Testing and Verification Approach

The framework was verified at three levels that for a research artefact corresponds to the unit, functional and acceptance testing of a conventional application. At the component level, which is analogous to white box unit testing, each layer was checked against a property that its design implies. The consensus counters were checked to conserve transactions so that committed, rule rejected and policy rejected counts sum to the input, the integrity computation was checked against hand worked cases, the Kernel SHAP implementation was checked to respect the additivity property in that the attributions plus the base value recover the model output, and the block level test was checked to behave correctly at its extremes. A particularly informative component check is that the deterministic rule layer alone stops almost no attacks, which is not a fault but the expected consequence of attacks that use registered identifiers and well formed payloads, and which therefore validates both the contract logic and the realism of the datasets.

At the functional level (which is analogous to black box testing) the framework was run end to end on both datasets and confirmed to produce every artefact the requirements demand (measured consensus indicators, trained detectors, per instance and global explanations and language model outputs) with no run time errors and with the earlier stages' results unchanged when the later stages were added. Reproducibility was verified by rerunning the

notebook from the openly available data and obtaining identical results, since every random process is seeded and the configuration is fixed. At the acceptance level, the framework was exercised not on synthetic inputs but on real, labelled attack traffic from two physical testbeds so that the evaluation reflects realistic operating conditions rather than contrived ones. Finally, the framework was assessed against the requirements of Section 4.1.1 and found to meet them as the functional requirements are evidenced by the results in the following sections, and the nonfunctional requirements of measurability, comparability, control and reproducibility are satisfied by the measured KPIs, the equal sized streams, the seeded averaging and the single rerunnable notebook respectively.

5.2 Consensus and Smart Contract Results

The measured consensus indicators are reported in Table 1 and visualised in Figure 1 and they answer RQ1. The first and clearest result is that integrity is governed by protocol structure rather than by the traffic is that the two datasets yield almost identical integrity for every algorithm, for example 88.2% against 88.4% for Proof of Work. The Byzantine and authority-based protocols keep every attack off the ledger, with Proof of Authority, Practical Byzantine Fault Tolerance and delegated Byzantine Fault Tolerance all reaching 100% integrity, because an audit or a two thirds quorum overrules a malicious proposer. The slashing based economic protocols recover most integrity, with Proof of Stake and Delegated Proof of Stake reaching 98.2% and 96.7%, while Proof of Work and Proof of Capacity, which have no such deterrent, admit approximately 12% of attacks. The most instructive single figure is Raft, the crash fault tolerant leader protocol, which admits approximately 8% of attacks at an integrity of 92.1% because a lying leader simply replicates a corrupt log to its followers. This demonstrates empirically that fault tolerance and attack resistance are distinct properties, a distinction that the consensus surveys reviewed in Section 2.2 describe qualitatively but do not quantify on domain traffic.

This security is bought at a communication and energy cost that Table 1 makes concrete. Raft needs only 13 messages per block, whereas delegated Byzantine Fault Tolerance needs 860, and Proof of Work incurs by far the largest modelled energy at 4150 relative units owing to its real hashing effort. The security to cost tradeoff is visualised Figure 1.6, which separates the cheap but weak leader protocols from the secure but expensive Byzantine protocols. A notable smart contract result answers RQ2 and takes the form of an ablation and it is that with the learned policy removed the deterministic rule layer alone stops essentially no attacks (approximately 0% for IoT and 1.2% for UAV) because the attacks use registered identifiers, legal data length codes and plausible timing. The entire burden of realistic detection therefore falls on the anomaly policy, which is the empirical justification for the AI layer that follows.

5.3 AI Misbehaviour Detection Results

The record level detection results are reported in Table 2, the blockchain facing view in Table 3, the confusion matrices for the strongest detectors in Figure 3, the receiver operating characteristic analysis in Table 4 and Figure 4 and the block level results in Figure 5 altogether answer RQ3. Detection is markedly easier on the UAV data, where the tree ensembles separate the classes almost perfectly, with histogram gradient boosting reaching an F1 of 0.961 and an area under the curve of 0.997, than on the heterogeneous IoT telemetry the best model extra trees reaches an F1 of 0.848 and an area under the curve of 0.882. The linear and probabilistic models collapse on the UAV data to an F1 of approximately 0.48 because the decision boundary lies in combinations of payload bytes that a single hyperplane cannot express, a point that the explainability analysis in Section 5.4 makes precise.

The blockchain facing view in Table 3 is sharper still and constitutes a second ablation, contrasting the two error costs that the ledger makes meaningful. Gaussian naive Bayes attains a 0% false negative rate on both datasets, missing no attack, yet is near useless because it rejects most valid traffic, with a false positive rate of 73.5% on IoT and 83.7% on UAV with AdaBoost behaves similarly on IoT. No single model optimises both security, measured as the false negative rate, and availability, measured as the false positive rate. The tree ensembles offer the best balance, for example histogram gradient boosting on UAV with a false negative rate of 5.7% and a false positive rate of only 1.1%, whereas the same model on the harder IoT data trades a 7.5% false negative rate against a 44.2% false positive rate. This tension between missed attacks and rejected valid records is exactly the operational tradeoff that a security designer must resolve, and it is visible only because detection is evaluated in blockchain terms rather than by accuracy alone. Lifting detection from the record to the block the unit that consensus commits removes much of this difficulty: as Figure 5 shows, the abnormal block test reaches 100% accuracy on both datasets because even an imperfect record level detector flags an attack heavy block reliably once the evidence compounds over its hundred records. The receiver operating characteristic ranking in Table 4 confirms the record level picture in a threshold independent form (with the tree ensembles outstanding on UAV) up to an area under the curve of 0.997, and strong on IoT at around 0.88 to 0.89, while the linear models fall to approximately 0.77 on UAV, quantifying in a single number that the UAV attack signature is non-linear.

5.4 Explainability and the Key Interpretive Finding

The global feature importances for the strongest detector on each dataset are shown in Figure 2, and they answer RQ4. On the IoT data, detection is driven by sensor state

indicators such as the smartphone signal, the door state and the temperature condition, which is unremarkable. However, on the UAV data the leading features are the CAN identifier and the payload bytes, whereas the inter frame timing ranks only 26 of 30 features. This is the study's key interpretive result. Although controller area network attacks are intuitively assumed to be floods, which would make timing the discriminating feature, SHAP shows that the detector keys almost entirely on payload content, which means that the attacks are protocol conformant payload injections consistent with the fuzzy and replay scenarios in the source dataset [6]. This single finding explains the entire pipeline as it is precisely why the deterministic contract rules (which check the registry) the data length code and the rate limit, all of which the attack satisfies, catch almost nothing, therefore why a learned and explainable detector is necessary. LIME corroborates the leading features with a top ten agreement of 40% on UAV and 20% on IoT, the partial overlap is expected given the different sampling of the two methods, so SHAP is treated as primary and LIME as corroboration. The explainability layer therefore does genuine work rather than merely ranking features since it overturns a modelling assumption and in doing so justifies the architecture, which is the standard by which Section 2.4 argued XAI in security should be judged.

5.5 LLM Assisted Interpretation Results

The behaviour of the language model stage is reported in Table 5 and Figure 6, and it answers RQ5. All three local models produce usable output. The modified code correctness of the generated smart contract rules is high across the board (between 0.900 and 0.950) and the semantic fidelity of the explanations, measured by BERTScore against a domain reference, is tightly clustered between 0.819 and 0.842, which indicates that even small open models running locally can turn an explanation into a coherent justification and a valid candidate rule. The more interesting result concerns retrieval augmentation, and it takes the form of a third ablation that contrasts the without retrieval and with retrieval settings. Contrary to the expectation set by the source literature, retrieval augmentation does not improve the output of these small local models as code correctness is marginally lower with retrieval than without it for all three models and BERTScore moves by amounts that fall within the 95% confidence intervals, for example from 0.819 to 0.823 for gemma2:2b. However, retrieval does reliably increase the response time, most clearly for llama3.2, which rises from 5.7 to 8.1 seconds, and for qwen2.5:3b, which rises from 12.2 to 14.1 seconds because the retrieved context lengthens the prompt. The honest reading, which Section 5.7 develops, is that for small local models on this task retrieval augmentation adds latency without a measurable quality benefit and that the differences between the models and settings are largely within confidence intervals, so the three models are best described as comparable rather than ranked.

5.6 Comparison with Existing Research

The results can now be compared with the literature reviewed in Chapter 2, and the comparison clarifies what is novel. On consensus, the surveys of blockchain for the Internet of Drones and of consensus protocols [1], [2], [13] describe the security and cost of these mechanisms qualitatively; the present work measures them on real IoT and UAV traffic and so converts assertions into figures, confirming for example that Byzantine protocols maximise integrity but at quadratic messaging cost and adding the empirical and less obvious result that a crash fault tolerant protocol admits roughly 8% of attacks. This measured, domain specific benchmark is the first contribution that distinguishes the work from prior blockchain literature, which does not couple consensus to a learned detector at all.

On detection, the deep learning intrusion detection survey of Ferrag et al. [16] and the in-vehicle controller area network detector of Song et al. [17] provide the closest baselines. The present detectors reach comparable strength on controller area network traffic, with near perfect separation echoing Song et al., but two differences are decisive. First, detection here is coupled to the ledger and evaluated in blockchain terms, through the false negative and false positive rates and the block level test rather than as a standalone classifier, which is exactly the integration that Chapter 2 found lacking. Second, the decisions are explained, where Wang et al. [19] applied SHAP to intrusion detection on a conventional network dataset as an after the fact audit, the present work applies LIME and Kernel SHAP to IoT and UAV detectors and uses the resulting finding (that the UAV attacks are payload injections rather than floods) as a design input that justifies the architecture, which is a stronger and more novel role for explainability than in the prior work.

On the language model stage, the comparison with the source studies [9], [10] is the most nuanced and the most interesting. Chatzimiltis et al. report that retrieval augmentation improves the modified code correctness of large hosted models on a radio access network compliance task. Adapting their methodology to blockchain secured IoT and UAV data and to small open models running locally, the present work finds that this benefit does not transfer as retrieval augmentation does not measurably improve correctness or semantic fidelity for all three local models and it increases latency. This is a genuine and defensible contribution to knowledge since it outlines a boundary on a published result by showing that the advantage of retrieval augmentation depends on model scale and task, and it is only visible because the methodology was reproduced faithfully in a new setting. Taken together, no prior system combines a measured consensus benchmark, ledger coupled and explainable detection, and a language model interpretation stage on real IoT and UAV

traffic, which is the overall novelty claimed for the framework and validated in part by the peer reviewed conference paper [11] on which its first two parts rest.

5.7 Critical Evaluation of Results

The results are strong but must be qualified honestly. The most important limitation concerns the consensus measurements. The block times in Table 1 are the compute times of the simulation not the network latencies of a deployment, the energy is a modelled relative quantity and the adversary fraction is assumed rather than observed. The consequence is that the absolute latency and energy figures should not be read as deployment values, although the relative ordering across protocols, which is what the study uses, remains valid because all protocols are measured under identical conditions. In contrast, the integrity and messaging figures are exact counts and carry no such caveat.

On detection, the high false positive rate on the harder IoT data (for example 31.2% for extra trees and 44.2% for histogram gradient boosting) is a real availability cost that the block level aggregation mitigates but does not eliminate at the record level, and it reflects the genuine heterogeneity of the IoT telemetry rather than a modelling error. On explainability, the from first principles Kernel SHAP is an approximation based on sampled coalitions, and LIME explanations are known to be less stable, which is precisely why the two are cross checked rather than relied upon singly. On the language model stage, three caveats matter and were surfaced during development. The code correctness rubric saturates near its ceiling because generating a small, well-formed rule is an easy task for modern models, so it discriminates weakly. Secondly, BERTScore is computed against an automatically generated domain reference rather than a human written one, so it measures adherence to a domain informed template rather than to a human gold standard. Finally, the stage uses six sampled attacks per dataset, which yields the wide confidence intervals seen in Table 5 and means that the small differences between settings should not be over interpreted. Finally, the whole study rests on two specific datasets, and although they are realistic and widely used, generalisation to other IoT and UAV deployments is an assumption rather than a demonstrated fact. These limitations motivate several of the future directions in Chapter 6.

5.8 Critical Evaluation of Project Objectives

Each objective from Section 1.2 can now be assessed against the evidence. The first objective, to review and critically evaluate the state of the art and identify the gaps, was met in Chapter 2, which produced a critical comparison, three explicit gaps and five research questions, and which the results of this chapter have answered in turn. The second objective, to select suitable open and labelled datasets, was met through the

adoption of TON_IoT and UAVCAN [5], [6] and their preparation as equal sized transaction streams. The third objective, to design and implement a unified framework that benchmarks eight consensus algorithms and two contracts on measured indicators, was met and is evidenced by Table 1 and Figure 1, and it answered RQ1 and RQ2. The fourth objective, to integrate ten detectors at the record and block level and to explain them with LIME and Kernel SHAP, was met and is evidenced by Tables 2,3 and 4 and Figures 2,3,4 and 5, and it answered RQ3 and RQ4 including the key interpretive finding. The fifth objective, to add and evaluate a language model interpretation stage with and without retrieval augmentation across several models was met and is evidenced by Table 5 and Figure 6, and it answered RQ5 with the honest and valuable finding that retrieval augmentation does not benefit small local models on this task. The sixth objective, to evaluate the framework end to end and to discuss the findings critically against the aim, the objectives and the literature, is met by this chapter as a whole. The objectives were therefore achieved in full, and the aim to design and empirically evaluate an explainable blockchain and AI framework that lets designers choose mechanisms on measured evidence is correspondingly satisfied.

5.9 Summary

This chapter has tested, presented and evaluated the framework. Verification at the component, functional and acceptance levels confirmed that the implementation realises the design and runs correctly on real traffic from two testbeds, and that it meets its requirements. The results answered the five research questions:

- consensus security is governed by protocol structure, with Byzantine and authority protocols keeping all attacks off the ledger and a crash fault tolerant protocol admitting roughly 8% at far lower cost.
- the deterministic rules alone stop almost nothing, so a learned policy is required.
- the tree ensembles give the best balance of security and availability and reach full accuracy at the block level.
- explainability overturns the assumption that the UAV attacks are floods, revealing protocol conformant payload injections and thereby justifying the architecture.
- small local language models produce faithful justifications and valid rules, but retrieval augmentation does not measurably help them.

The comparison with existing research established the novelty of the integrated, measured and explainable pipeline and delineated a boundary on a published retrieval augmentation result, and the critical evaluation stated the simulation and evaluation limitations honestly while confirming that all objectives were met.

The final chapter draws the dissertation to a close. Chapter 6 revisits the aim and objectives in the light of these results, summarises the contributions, reflects on the limitations identified here, and sets out directions for my potential future work that will follow directly from them.

CHAPTER 6: CONCLUSION

This final chapter concludes the dissertation. Section 6.1 summarises the work and explains how each objective was met and how, in meeting them, the overall aim was achieved. Section 6.2 presents the research contributions and discusses how they can be exploited both academically and practically, together with the main findings and insights the project produced. Section 6.3 acknowledges the limitations of the study and turns each of them into a concrete direction for future research and development. Section 6.4 offers a personal reflection on the strengths and weaknesses that completing the project revealed and on the steps that would address those weaknesses in future.

6.1 Summary of the Dissertation

The dissertation addressed a problem that recurs across IoT and unmanned aerial vehicle security and it is how to detect misbehaving nodes on the basis of measured rather than assumed evidence, explain each decision, and translate that explanation into an actionable and auditable response, all within one coherent and reproducible framework. Chapter 2 established through a critical review that no existing work combines these capabilities, distilling the field into three persistent gaps and five research questions, and the remaining chapters answered those questions in turn. The work was structured around six objectives, and it is the completion of these objectives together that achieves the aim.

The first objective, to review and critically evaluate the state of the art and to identify the gaps, was met in Chapter 2 and produced the comparison, the gaps and the research questions that governed the rest of the project. The second objective, to select suitable open and labelled datasets, was met through the adoption and preparation of the TON_IoT and UAVCAN datasets as equal sized transaction streams. The third objective, to design and implement a unified framework that benchmarks eight consensus algorithms and two smart contracts on measured indicators, was met and answered in the first two research questions: it showed that consensus security is governed by protocol structure, with Byzantine and authority based protocols keeping every attack off the ledger while a crash fault tolerant protocol admits roughly 8% at far lower cost, and it showed by ablation that the deterministic rules alone stop almost nothing, so a learned policy is required. The fourth objective, to integrate ten detectors at the record and block level and to explain them with LIME and Kernel SHAP, was met and answered the next two questions: the tree ensembles gave the best balance of security against availability and reached full accuracy at the block level, and the explainability layer produced the study's key finding, that the UAV attacks are protocol conformant payload injections rather than the assumed floods, which in turn justifies the whole architecture. The fifth objective, to add and evaluate a

language model interpretation stage with and without retrieval augmentation across several models, was met and answered the final question, with the honest result that small local models produce faithful justifications and valid candidate rules but that retrieval augmentation does not measurably improve them. The sixth objective, to evaluate the framework end to end and to discuss the findings critically, was met by Chapter 5.

Because each objective was achieved and each contributed a necessary component of the whole, the aim is satisfied because the project has designed and empirically evaluated an explainable blockchain and AI framework for detecting and justifying misbehaviour in IoT and UAV networks is satisfied, and it provides the measured evidence on which a designer can select consensus, detection and explanation mechanisms.

6.2 Research Contributions

The project makes several contributions, each of which has both an academic and a practical dimension. The first is a unified and reproducible framework that runs real consensus and smart contracts directly on real IoT and UAV traffic by mapping every record to a ledger transaction. Academically, this provides a common experimental basis on which consensus, detection and explanation can be studied together rather than in isolation, and its first two parts have already been validated through my co-authored and peer reviewed conference paper [11]. Practically, because the framework is expressed as a single, openly reproducible notebook that uses only free and local components, including local language models that require no paid interface, it can be adopted and extended by practitioners and by future students without cost or licensing barriers.

The second contribution is the measured consensus benchmark itself. Academically, it converts the qualitative claims of the consensus surveys [1], [2], [13] into figures for the IoT and UAV domain, and it contributes the empirical and somewhat counter intuitive insight that fault tolerance and attack resistance are distinct properties, evidenced by a crash fault tolerant protocol that admits attacks a Byzantine one rejects. Practically, it gives a system designer a direct, evidence-based answer to the question of which consensus family to choose for a given balance of security, communication cost and energy. The third contribution is the coupling of explainable detection to the ledger, evaluated in blockchain terms through the false negative and false positive rates and the block level test.

Academically, it demonstrates a stronger role for explainability in security than the after the fact auditing seen in prior work such as [19], because the SHAP finding is used as a design input; the insight that the UAV attacks key on payload content rather than timing is itself a transferable result about the nature of controller area network attacks. Practically, it gives an operator an auditable account of why any record was admitted to or kept off the ledger and a candidate mitigation rule to act upon.

The fourth contribution is the language model interpretation stage and the finding that delineates a boundary on a published result. Academically, by reproducing the methodology of [9] faithfully in a new domain and with small local models, the project shows that the benefit of retrieval augmentation reported for large hosted models does not transfer to this setting, which is a genuine addition to knowledge because it scopes the conditions under which retrieval augmentation helps. Practically, it shows that a usable explanation to mitigation stage can be built entirely from free, locally hosted models, which lowers the barrier to deploying such a capability. Taken together these contributions clearly present the project's central insight that treating security, trust and explanation as one measured pipeline, rather than as separate concerns yields findings that none of the parts reveals alone.

6.3 Limitations and Future Research and Development

No study is without limitations and acknowledging them honestly both strengthens the evaluation and points directly to future work. The most significant limitation is that the study rests on two datasets which, although realistic and widely used, cannot establish generalisation on their own. Future work should apply the framework to further IoT and UAV datasets and to additional attack types, which would test whether the payload injection finding and the detector rankings hold more broadly. A third limitation concerns detection as the false positive rate on the heterogeneous IoT data is high at the record level, and although block level aggregation mitigates it, future work could apply cost sensitive learning or richer temporal features to reduce it directly. A fourth concerns the explainability implementation, where Kernel SHAP is an approximation and LIME can be unstable, future work could incorporate exact or more heavily sampled attributions and a formal stability analysis. The language model stage carries the richest set of opportunities, each following from a limitation identified in Chapter 5. The code correctness rubric saturates because the generation task is easy, so a more demanding task in which the generated rule must be executable and is tested against sample transactions would discriminate more sharply and would also close the loop by feeding synthesised rules back into the smart contract automatically. The BERTScore evaluation uses an automatically generated reference, so a human referenced evaluation would strengthen it. The stage uses small local models and six sampled attacks per dataset, so larger models and a larger sample planned with a target confidence interval in mind would sharpen the comparison and might yet reveal a retrieval benefit that the small local configuration did not. Pursuing even a subset of these directions would build naturally on the artefact this project leaves behind.

6.4 Personal Reflections

Completing this dissertation has been demanding and has made me more aware of both my strengths and my weaknesses. My clearest strength is a willingness and an ability to work across several distinct fields at once. The project required me to understand distributed consensus and blockchain, classical machine learning, explainability and large language models, and to integrate them into one coherent system rather than treating them as separate exercises. Implementing the consensus mechanisms and the LIME and Kernel SHAP explainers from first principles, rather than relying on libraries, was difficult but it deepened my understanding of how these methods actually work in a way that using them as black boxes never would have. A second strength that grew over the project is empirical honesty, I learned to insist on distinguishing what was measured from what was modelled or simulated, and to label real results separately from placeholder ones, and I became more comfortable challenging outputs and verifying figures for myself rather than accepting them, which I regard as the most valuable research skill I have developed. Bringing the first two parts of the work to the standard of a coauthored, peer-reviewed conference paper was a rewarding confirmation of these strengths.

My weaknesses were also exposed. The first is scope and time management as my initial ambitions were larger than four months allowed, and I had to make scoping decisions under time pressure, such as omitting the agentic retrieval setting from the llm implementation. In future I would agree to a tighter scope with my supervisor earlier and protect more time for evaluation and writing. The second is statistical rigour as the language model stage used a small sample and produced wide confidence intervals and I should have planned the sample size against a target precision from the outset rather than settling it late. The third is reproducibility across environments, where I lost time to tooling and environment problems, and in the future I would pin dependencies and containerise the environment early so that the work runs identically everywhere. The fourth is technical writing (which I found harder than the technical work itself) I would seek structured feedback on my writing sooner and more often. Recognising these weaknesses concretely is itself one of the more useful outcomes of the project, and each of them suggests a clear and practical step that I intend to carry into future research.

References

- [1] Hafeez, S., Khan, A.R., Al-Quraan, M.M., Mohjazi, L., Zoha, A., Imran, M.A. and Sun, Y. (2023) 'Blockchain-assisted UAV communication systems: a comprehensive survey', *IEEE Open Journal of Vehicular Technology*, 4, pp. 558–580. doi: 10.1109/OJVT.2023.3295208.
- [2] Jain, A. et al. (2024) 'A walkthrough of blockchain-based Internet of Drones architectures', *IEEE Internet of Things Journal*, 11(21), pp. 34924–34944.
- [3] Aggarwal, S., Shojafar, M., Kumar, N. and Conti, M. (2019) 'A new secure data dissemination model in Internet of Drones', in *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, pp. 1–6. doi: 10.1109/ICC.2019.8761372.
- [4] Nakamoto, S. (2008) *Bitcoin: a peer-to-peer electronic cash system*. Available at: <https://bitcoin.org/bitcoin.pdf>.
- [5] Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A. and Anwar, A. (2020) 'TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems', *IEEE Access*, 8, pp. 165130–165150. doi: 10.1109/ACCESS.2020.3022862.
- [6] Kim, D., Song, Y., Kwon, S., Kim, H., Yoo, J.D. and Kim, H.K. (2022) *UAVCAN intrusion dataset* [Dataset]. IEEE DataPort. doi: 10.21227/fcyc-bb14.
- [7] Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) "'Why should I trust you?': explaining the predictions of any classifier", in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, pp. 1135–1144.
- [8] Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 4765–4774.
- [9] Chatzimiltis, S., Boloursaz Mashhadi, M., Shojafar, M., Debbah, M. and Tafazolli, R. (2026) 'Agentic AI for 6G: a new paradigm for autonomous RAN security compliance', *IEEE Communications Standards Magazine* (in press). doi: 10.1109/MCOMSTD.2026.3681685.
- [10] Chatzimiltis, S., Shojafar, M., Boloursaz Mashhadi, M. and Tafazolli, R. (2026) 'AI-on-RAN for cyber defense: an XAI-LLM framework for interpretable anomaly detection', *IEEE Transactions on Network Science and Engineering*, 13 (in press).
- [11] Abdelghani, Y. and Shojafar, M. (2026) 'Explainable blockchain-AI misbehaviour detection for IoT and UAV networks', in *Proceedings of the IEEE International Conference on Next Generation Communications (NextGCom)*. IEEE (in press).
- [12] Dorri, A., Kanhere, S.S., Jurdak, R. and Gauravaram, P. (2017) 'Blockchain for IoT security and privacy: the case study of a smart home', in *Proceedings of the IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE, pp. 618–623. doi: 10.1109/PERCOMW.2017.7917634.
- [13] Xiao, Y., Zhang, N., Lou, W. and Hou, Y.T. (2020) 'A survey of distributed consensus protocols for blockchain networks', *IEEE Communications Surveys & Tutorials*, 22(2), pp. 1432–1465. doi: 10.1109/COMST.2020.2969706.

- [14] Castro, M. and Liskov, B. (1999) 'Practical Byzantine fault tolerance', in *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, pp. 173–186.
- [15] Ongaro, D. and Ousterhout, J. (2014) 'In search of an understandable consensus algorithm', in *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, pp. 305–319.
- [16] Ferrag, M.A., Maglaras, L., Moschogiannis, S. and Janicke, H. (2020) 'Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study', *Journal of Information Security and Applications*, 50, 102419. doi: 10.1016/j.jisa.2019.102419.
- [17] Song, H.M., Woo, J. and Kim, H.K. (2020) 'In-vehicle network intrusion detection using deep convolutional neural network', *Vehicular Communications*, 21, 100198. doi: 10.1016/j.vehcom.2019.100198.
- [18] Abdel-Basset, M., Moustafa, N., Hawash, H., Razzak, I., Sallam, K.M. and Elkomy, O.M. (2022) 'Federated intrusion detection in blockchain-based smart transportation systems', *IEEE Transactions on Intelligent Transportation Systems*, 23(3), pp. 2523–2537. doi: 10.1109/TITS.2021.3119968.
- [19] Wang, M., Zheng, K., Yang, Y. and Wang, X. (2020) 'An explainable machine learning framework for intrusion detection systems', *IEEE Access*, 8, pp. 73127–73141.
- [20] Mehta, P., Gupta, R. and Tanwar, S. (2020) 'Blockchain envisioned UAV networks: challenges, solutions, and comparisons', *Computer Communications*, 151, pp. 518–538. doi: 10.1016/j.comcom.2020.01.023.
- [21] Salah, K., Rehman, M.H.U., Nizamuddin, N. and Al-Fuqaha, A. (2019) 'Blockchain for AI: review and open research challenges', *IEEE Access*, 7, pp. 10127–10149. doi: 10.1109/ACCESS.2018.2890507.
- [22] Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q. and Artzi, Y. (2020) 'BERTScore: evaluating text generation with BERT', in *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [23] Wirth, R. and Hipp, J. (2000) 'CRISP-DM: towards a standard process model for data mining', in *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining (PADD)*, pp. 29–39.
- [24] Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [25] Lewis, P. et al. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 9459–9474.

Ethical Approval



Shojafar, Mohammad Dr (Comp Sci & Elec Eng)

To: @ Abdelghani, Yousef H (PG/T - Comp Sci & Elec Eng)

🚩 Flagged

Dear Yousef,

I confirm that an ethics submission is not required as there is no data collected nor human participants within your project.

Best regards,
mohammad

Appendix

Project Minutes:

Meeting 1:

- discussed possible ideas and project direction
- multiple related papers given to read and study throughout April and may before starting work on dissertation after exams period.

Meeting 2:

1. create notebook with consensus algorithm (PoW, DPOs, PoS, etc.) and add misbehaviour block
2. Consider blockchain KPIs
3. Try to test the model in large-scale blocks to be capable of running over IoT large- scale devices.
4. Study additional IoT papers
5. update the block selection procedure over the consensus algorithm and smart contract

Meeting 3:

add all consensus algorithms + PoA PoC (proof of capacity) raft pbft/dbft Comparison between algorithms with graphing/data visual representaion - all blockchain metrics - with related kpis use my own data set for now. Start write up and add to it after every task.

Meeting 4:

1. Identify two open datasets related to IoT and UAV systems, and implement consensus algorithms and smart contracts on top of them, supported by clear visual results.
2. Integrate AI-based misbehaviour detection into the system to identify malicious or abnormal blocks using selected IoT or UAV models. Use up to ten recent classical AI algorithms and visually compare their performance using confusion matrices and related Blockchain and AI KPIs .
3. Consider targeting the NextCom conference with a six-page IEEE-format paper, noting that the submission deadline is 15 August 2025. WWW.nextgcom.net

Meeting 5:

1. Identify ONE open datasets related to UAV systems, and implement consensus algorithms and smart contracts on top of them, supported by clear visual results.
2. Implement XAI using LIME/SHAP over IoT and UAV datasets with related results.
3. Consider targeting the NextCom conference with a six-page IEEE-format paper, noting that the submission deadline is 15 August 2025. WWW.nextgcom.net

Meeting 6:

- tune up code → roc area under curve average table and remove some unnecessary tables in the end
- clean up comments
- write up paper for nextgcom conference and add to overleaf to work on it with professor.

Meeting 7:

Improve conference paper by adding to related works in lit review section as well as more details results section, then submit paper

Meeting 8:

- add llm using papers provided based on table 2 on the main paper add RAG and no-RAG with different AI models. Then compare results using avg modified code correctness, avg response time and avg bert score (should be between 0.4-0.8)
- work heavily on main diss paper

Meeting 9:

- Code adjustments
- Finishing off paper

Meeting 10:

- Final submission go ahead